KU LEUVEN

Lecture 4: Ensembles

Machine learning and inductive inference

Sieben Bocklandt

Slides based on slidedecks from Jesse Davis' lectures in Principles of Machine Learning

Goals of this lecture

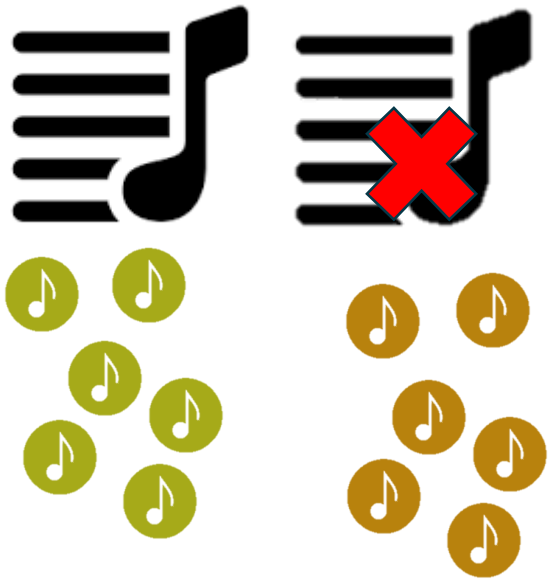
- Deep understanding of how ensemble algorithms work
  - Voting
  - Stacking
  - Bagging
  - Random forest
  - Boosting
  - Error-coding output codes
- Understand strengths and weaknesses of the ensemble methods
- Understand and be able to explain why ensembles work

Outline

- Voting, stacking, bagging and random forests
- AdaBoost, XGBoost
- Manipulating inputs and outputs
- Why ensembles work
- Applications

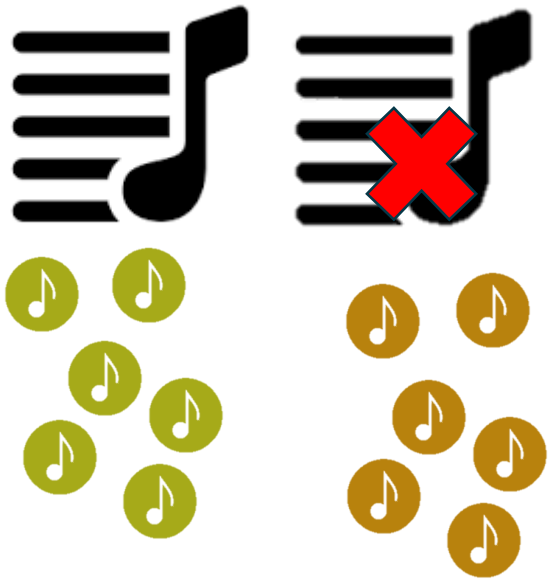
Running example

- Music playlist consisting of songs used as background music



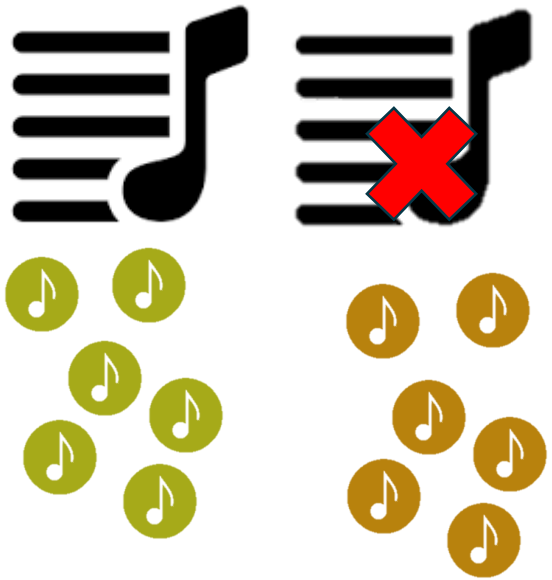
Running example

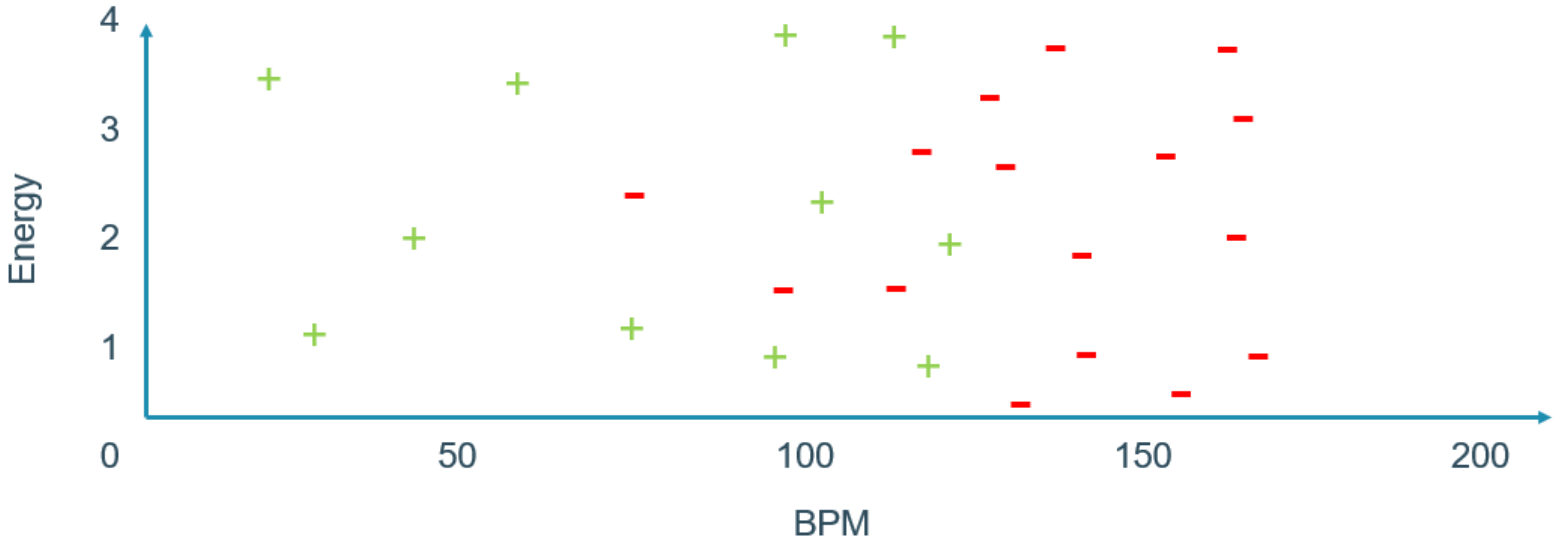
- Music playlist consisting of songs used as background music
- We want to automatically add new songs



Running example

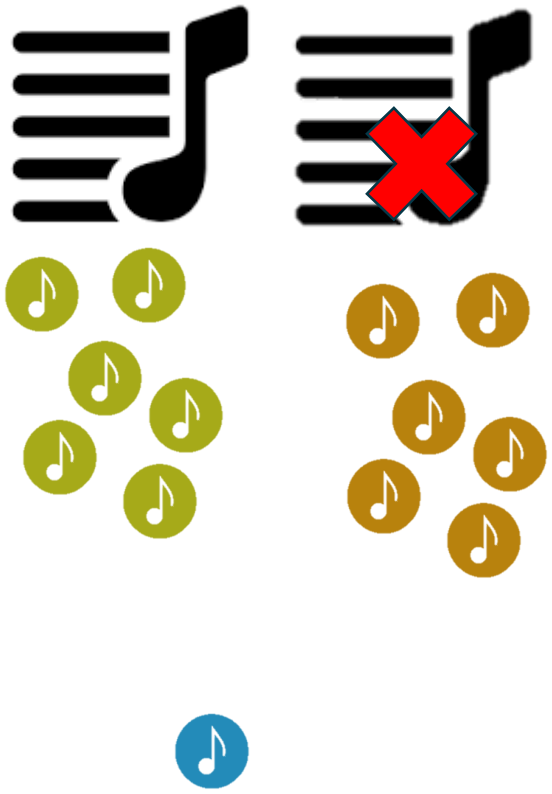
- Music playlist consisting of songs used as background music
- We want to automatically add new songs

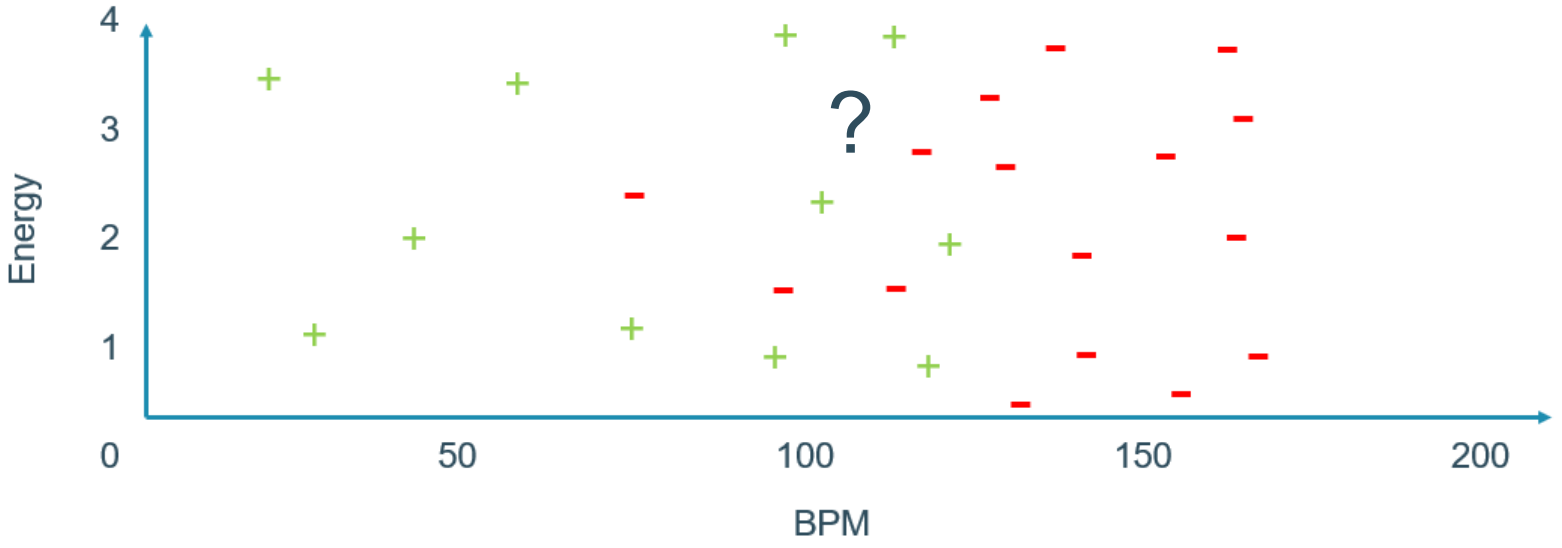




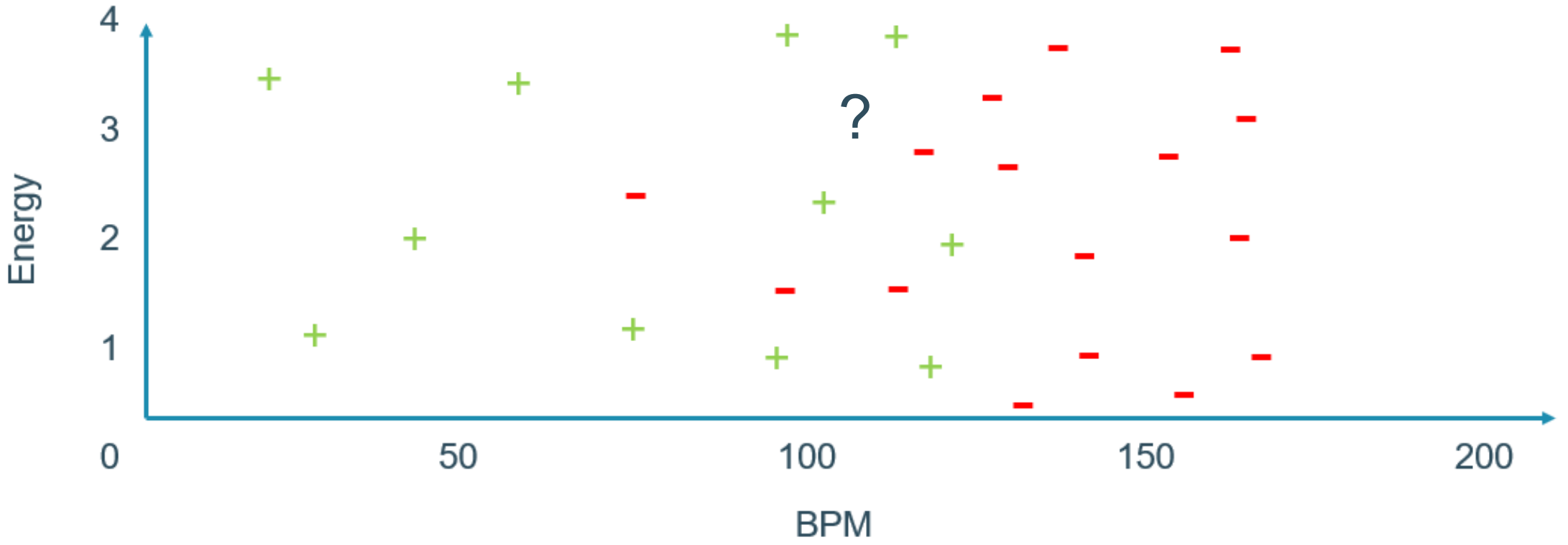
Running example

- Music playlist consisting of songs used as background music
- We want to automatically add new songs

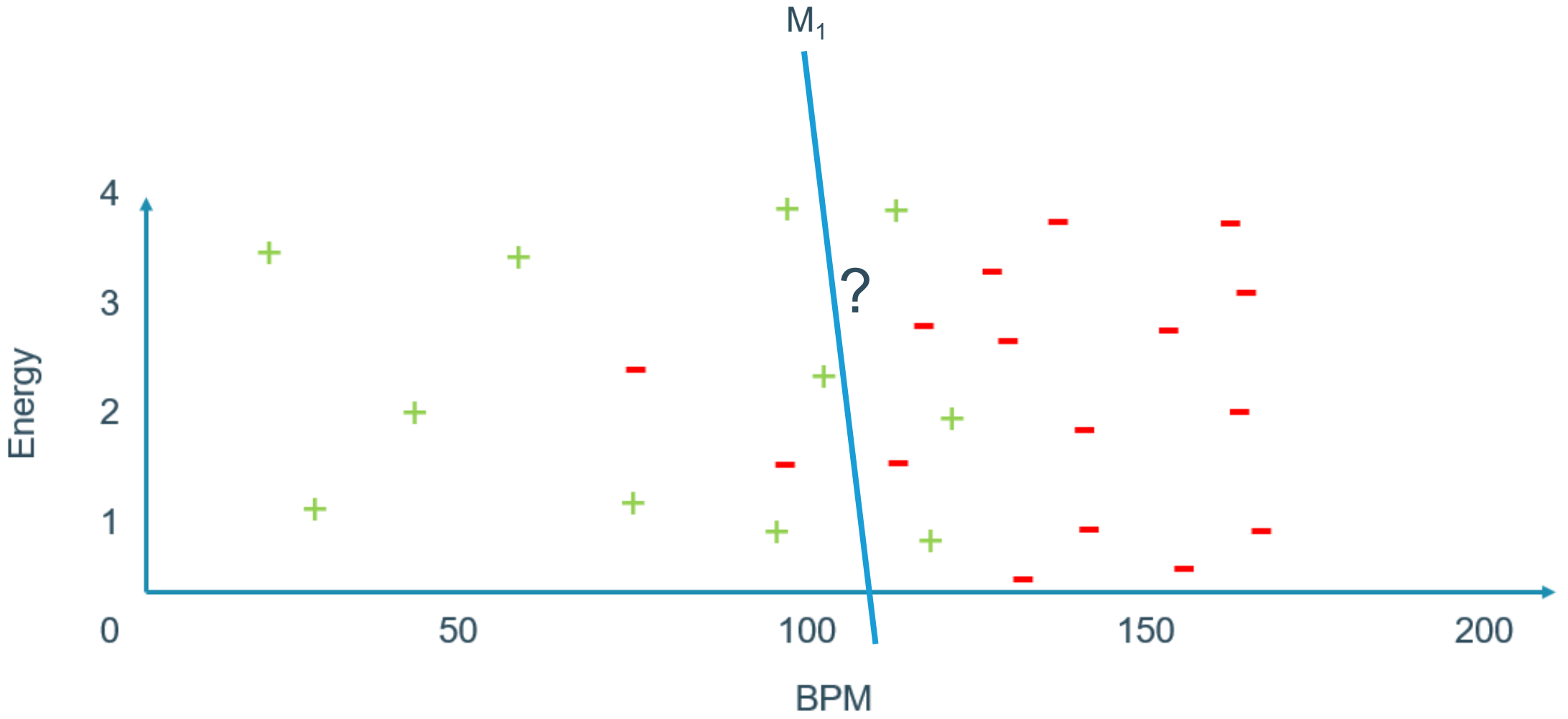




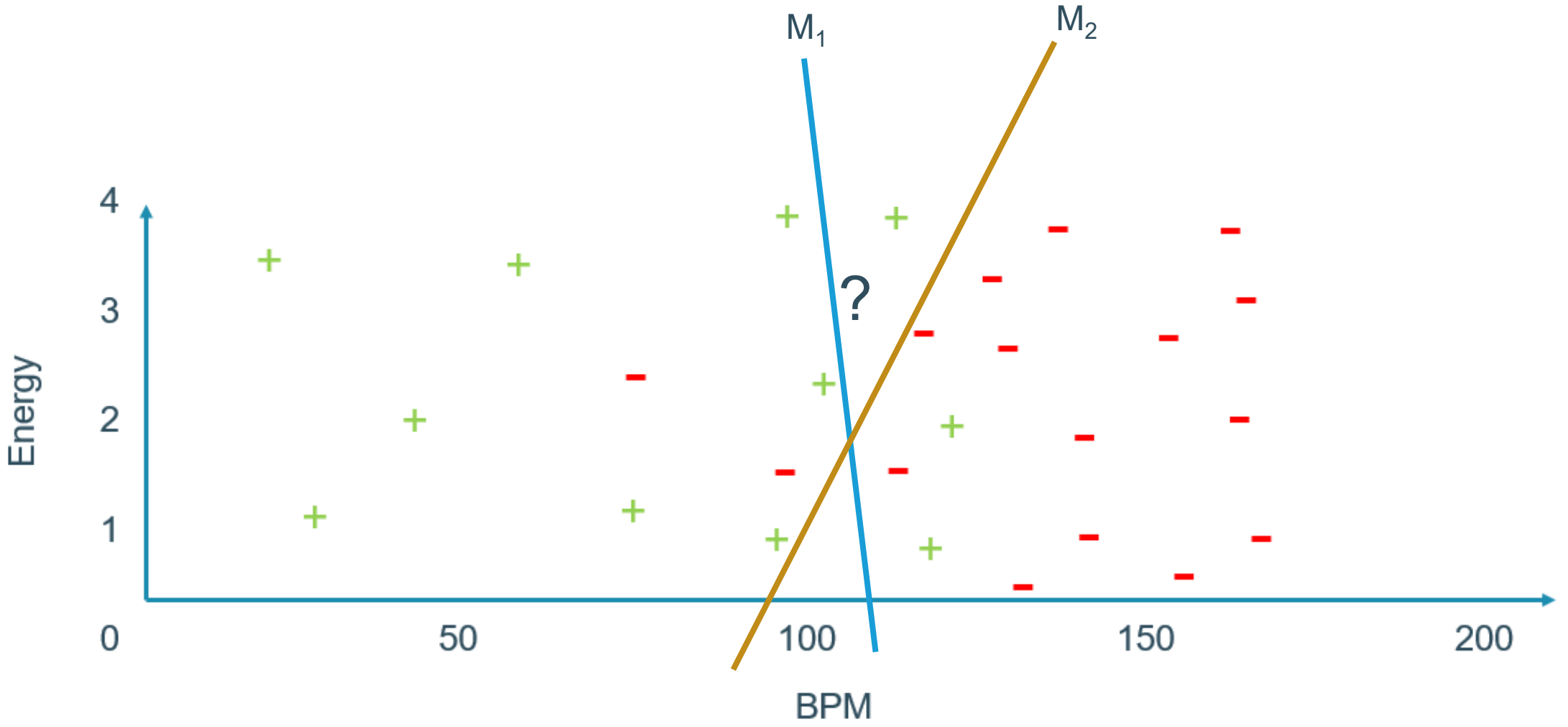
Running example: One model vs multiple models



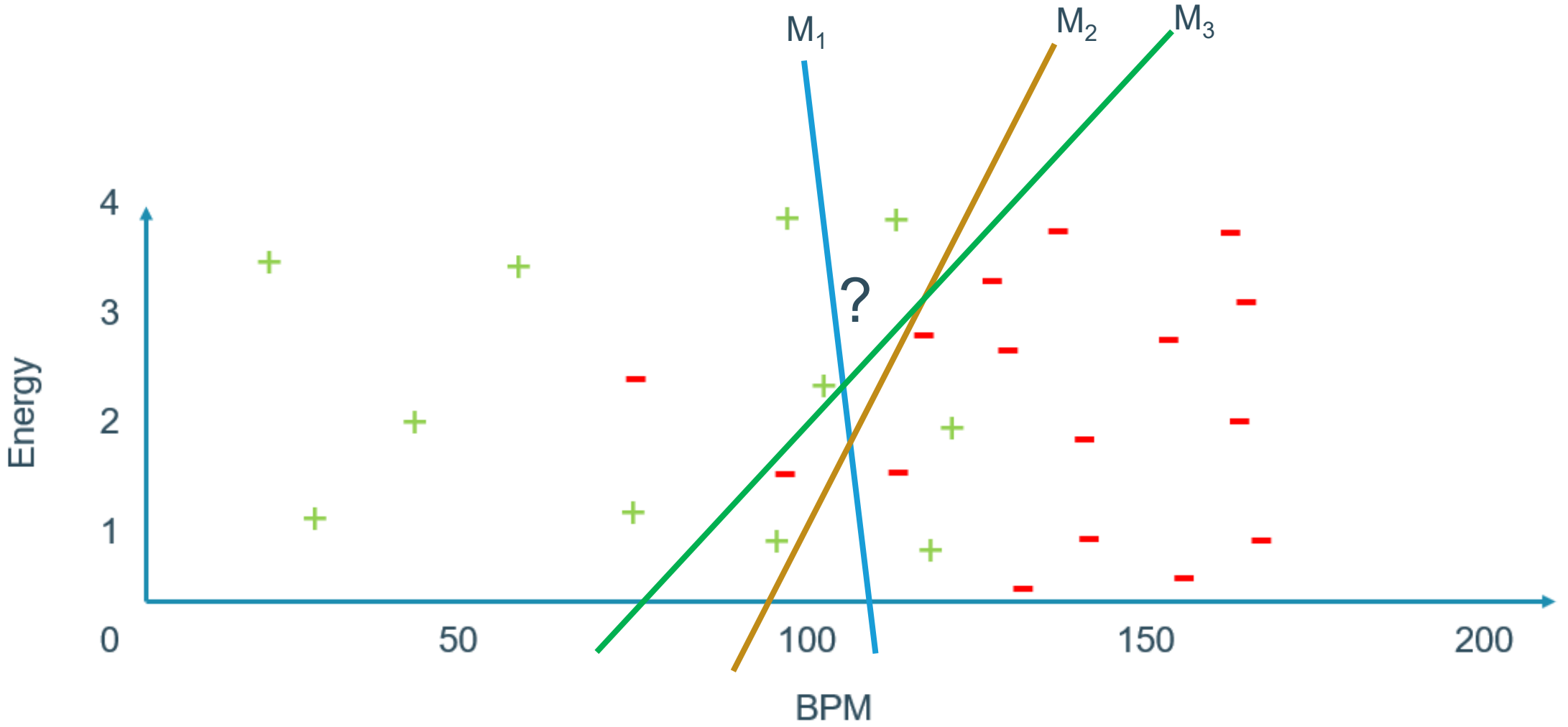
Running example: One model vs multiple models



Running example: One model vs multiple models



Running example: One model vs multiple models



Ensemble method

- An ensemble model is a predictive model that is a combination of several different predictive models.
  - Same type of models → homogeneous ensemble
  - Different type of models → heterogeneous ensemble
- An ensemble is often more accurate than any of its constituent models
- Why? Because “two know more than one”

Ensemble method

- An ensemble model is a predictive model that is a combination of several different predictive models.
  - Same type of models → homogeneous ensemble
  - Different type of models → heterogeneous ensemble
- An ensemble is often more accurate than any of its constituent models
- Why? Because “two know more than one”

But....

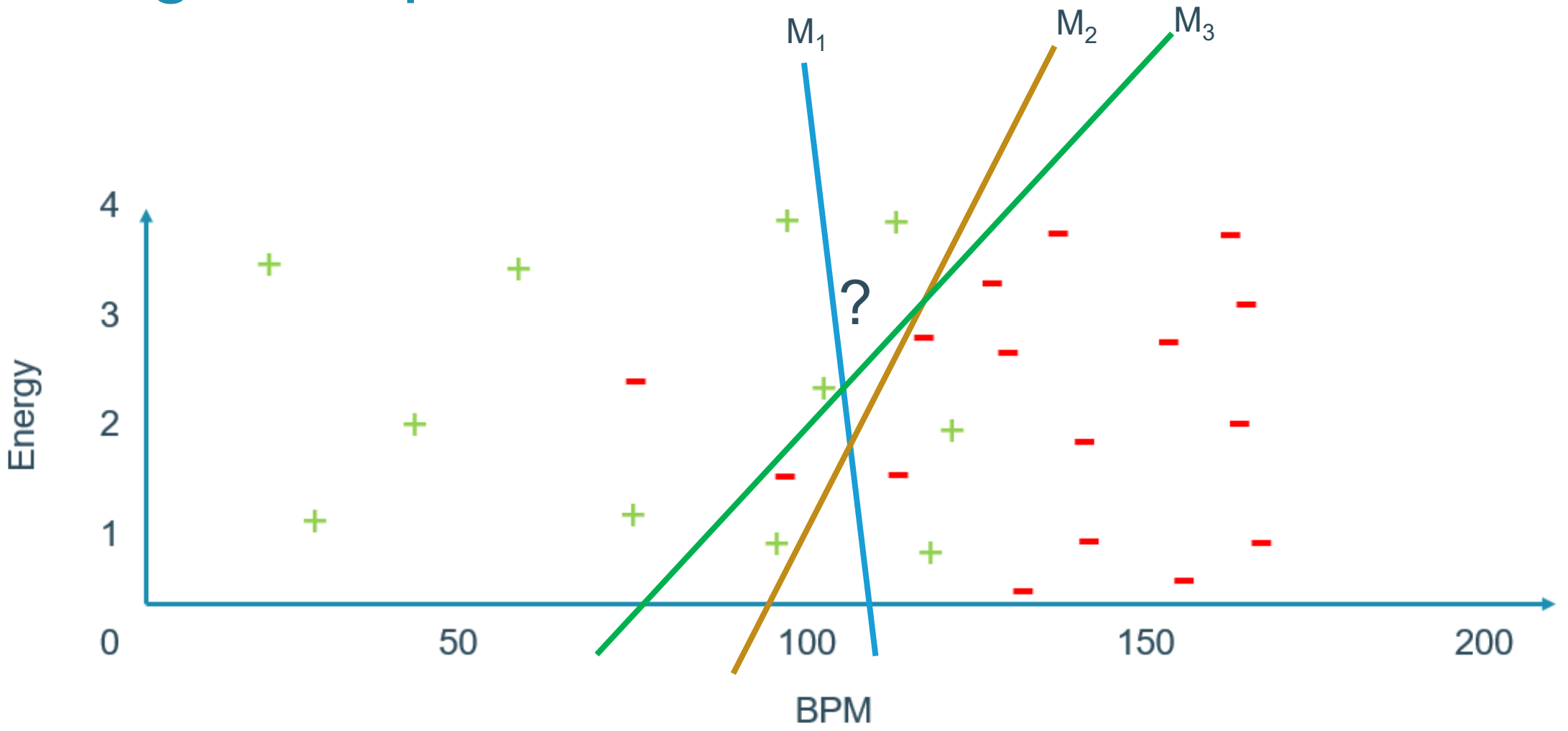
- How do we get multiple predictive models that make other mistakes?
- What models can we consider for this?
- How do we combine their predictions?

Use multiple models

Voting

- Assume we have k classifiers M_1, M_2, …, M_k
- Consider classifier M:
  - given a new instance x, M computes $M_1(x)$, $M_2(x)$, …, $M_k(x)$, counts how often each class is predicted, and returns the class that is predicted most often
  - In other words: M lets all classifiers vote and returns the majority vote as its answer
  - How well will M work?

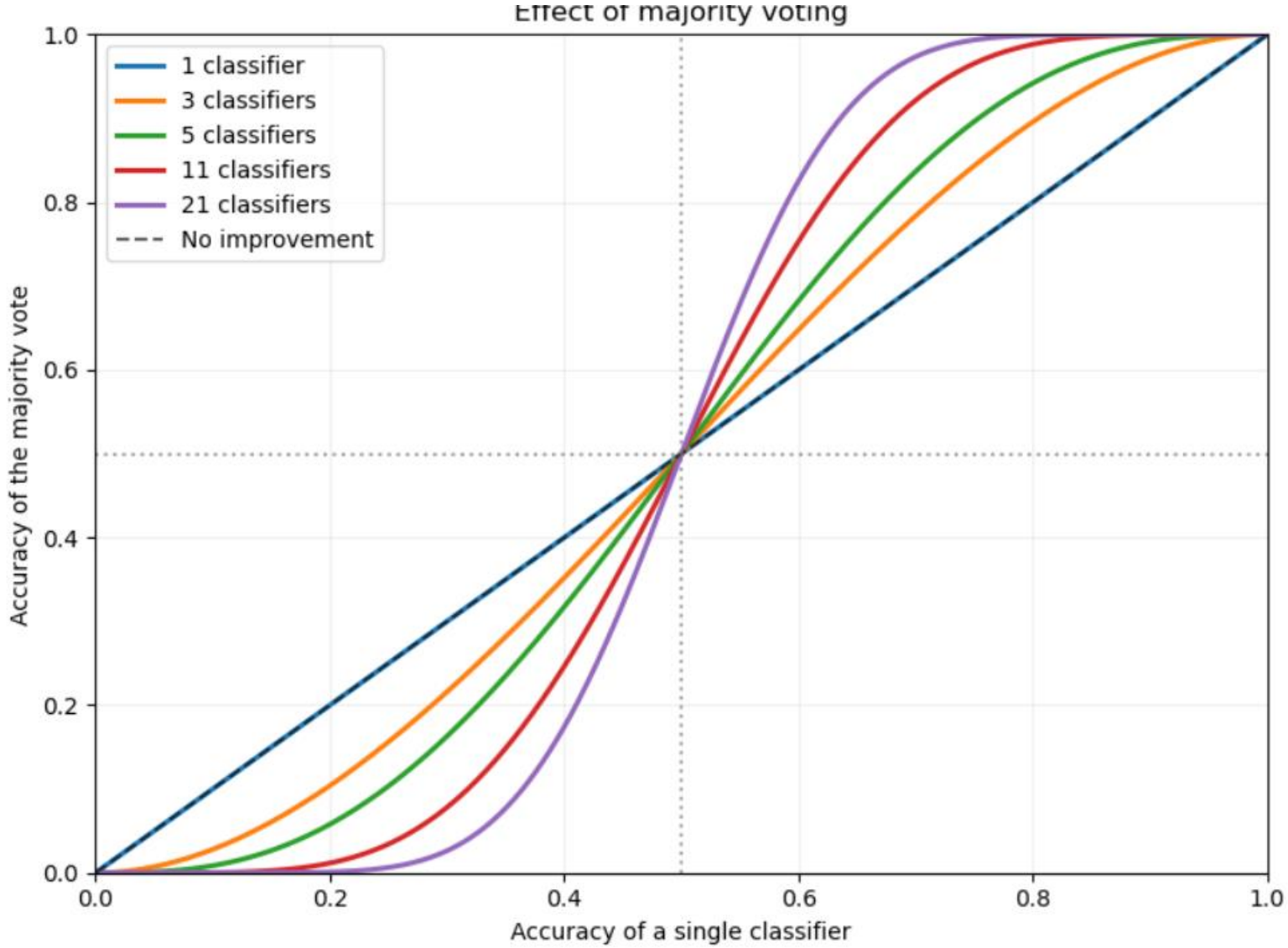
Voting example on music



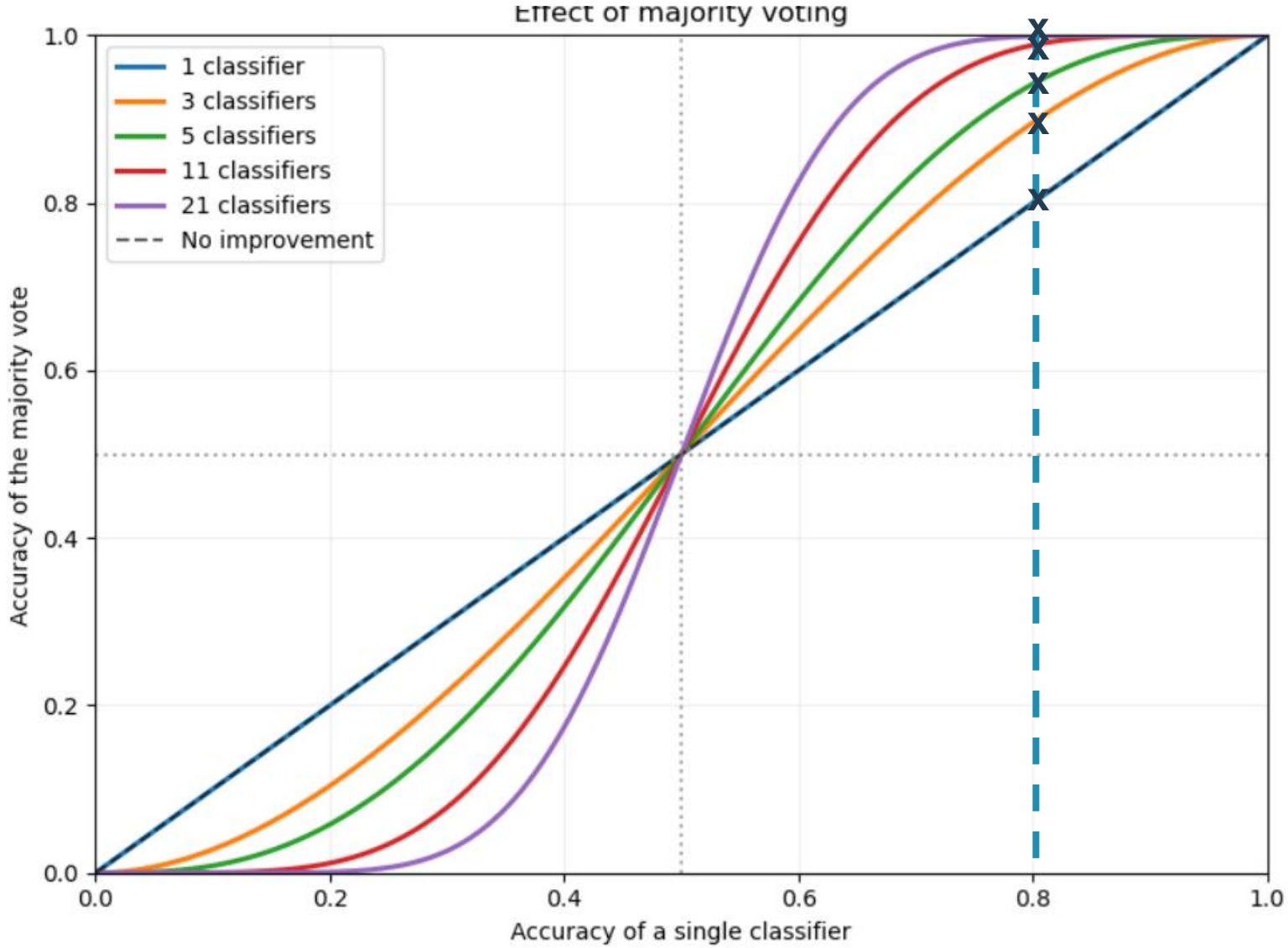
Voting

- The question is difficult to answer in general, but we can analyze what happens under some simplifying assumptions
- Let us assume:
  1. all classifiers M_1, …, M_k have the same accuracy *p*
  2. mistakes made by different classifiers are statistically independent
     - That is: for i≠j, P(M_i wrong | M_j wrong) = P(M_i wrong | M_j right) = P(mi is wrong)
     - Or: “if something is difficult for M_j, that doesn’t mean it’s also difficult for M_i ”

Voting


Effect of majority voting
1 classifier
3 classifiers
5 classifiers
11 classifiers
21 classifiers
No improvement
Accuracy of the majority vote
Accuracy of a single classifier
0.0
0.2
0.4
0.6
0.8
1.0

Voting



21 classifiers: 0.999

11 classifiers: 0.988

5 classifiers: 0.942

3 classifiers: 0.896

1 classifier: 0.800

Voting

- We can prove that voting helps if we have equally good models that make mistakes independently
- But what if:
  - mistakes are not independent?
  - some models are better than others?
  - ➔ Need experimental validation

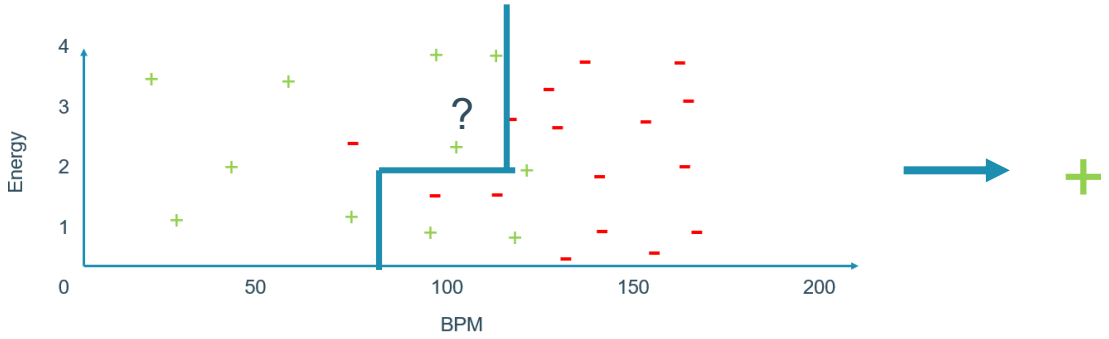
Stacking

– Use the votes as the input of a new learner

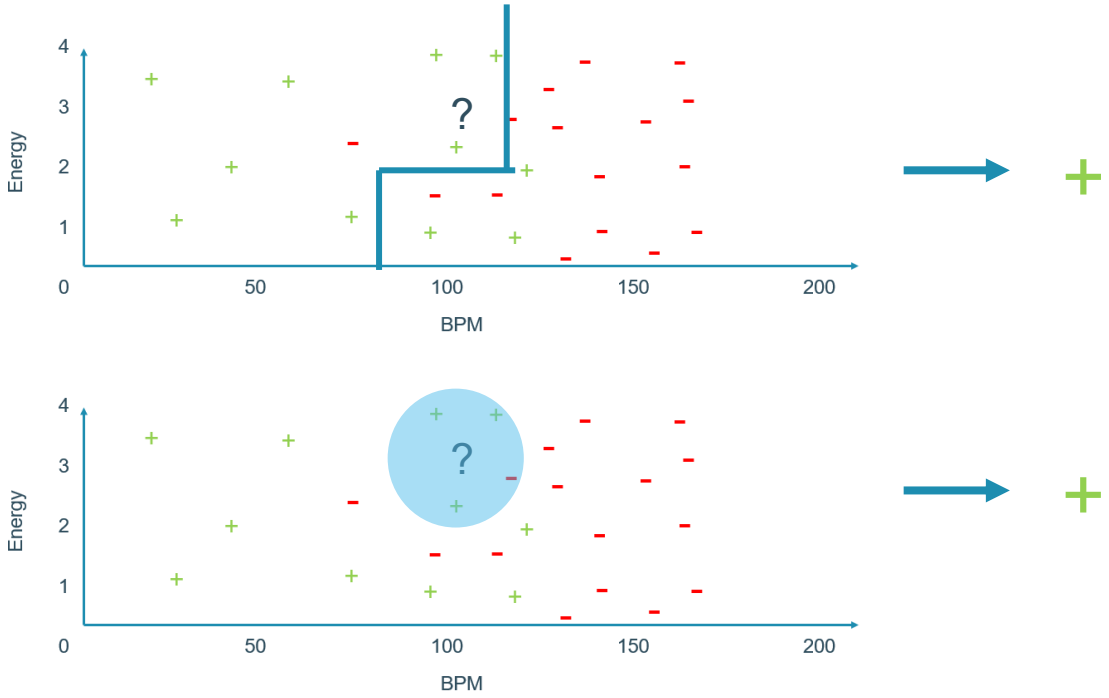
– This can also be seen as a data transformation



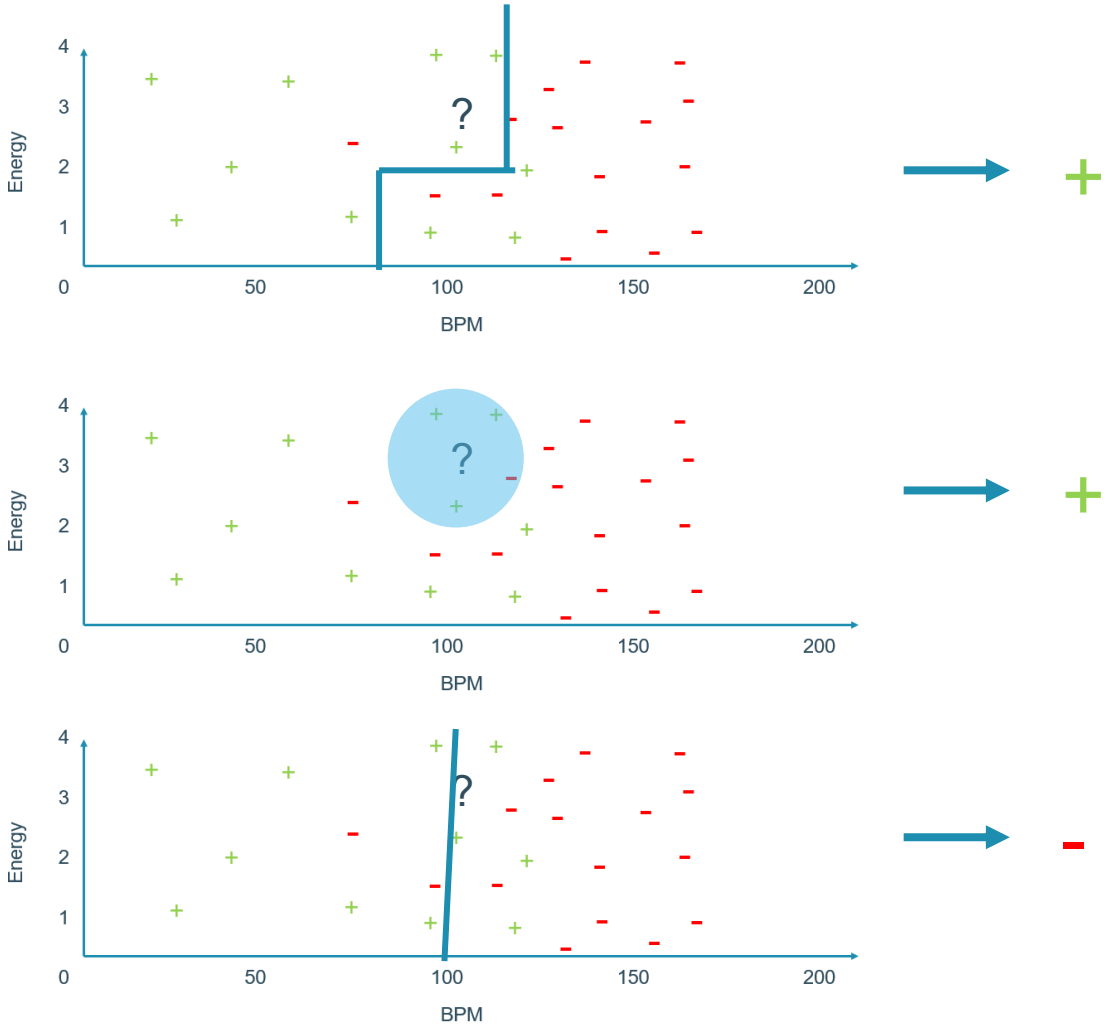
Stacking example on music



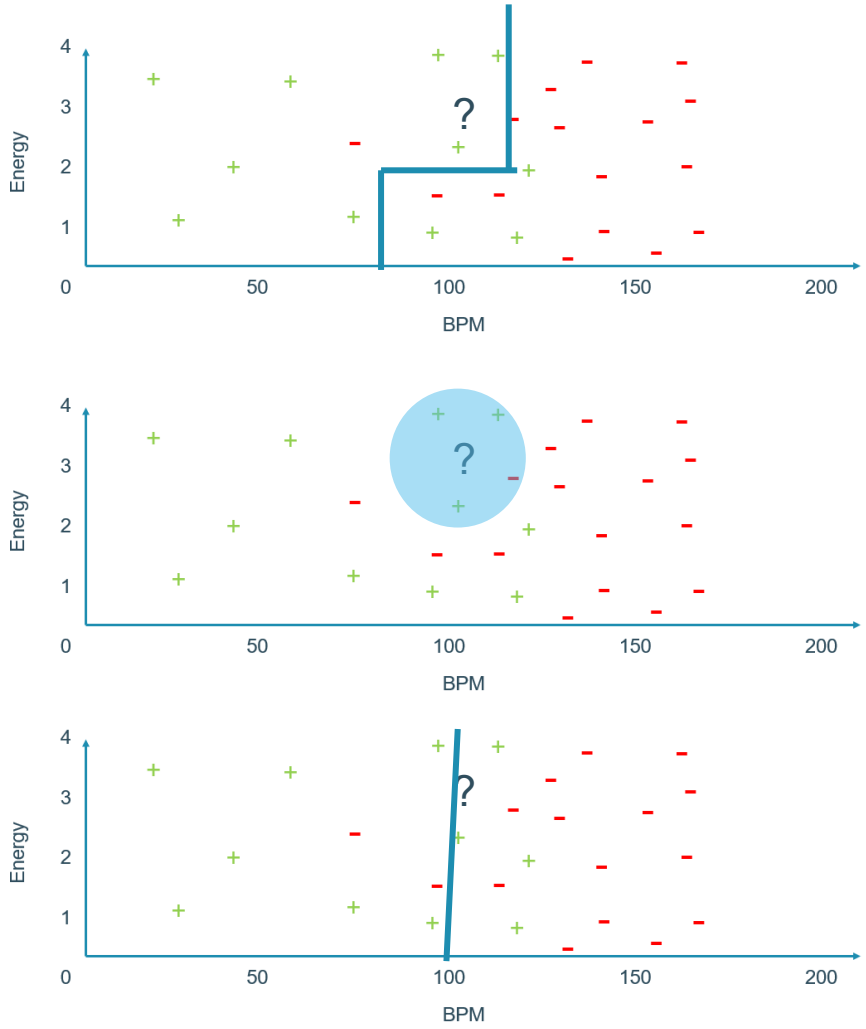
Stacking example on music

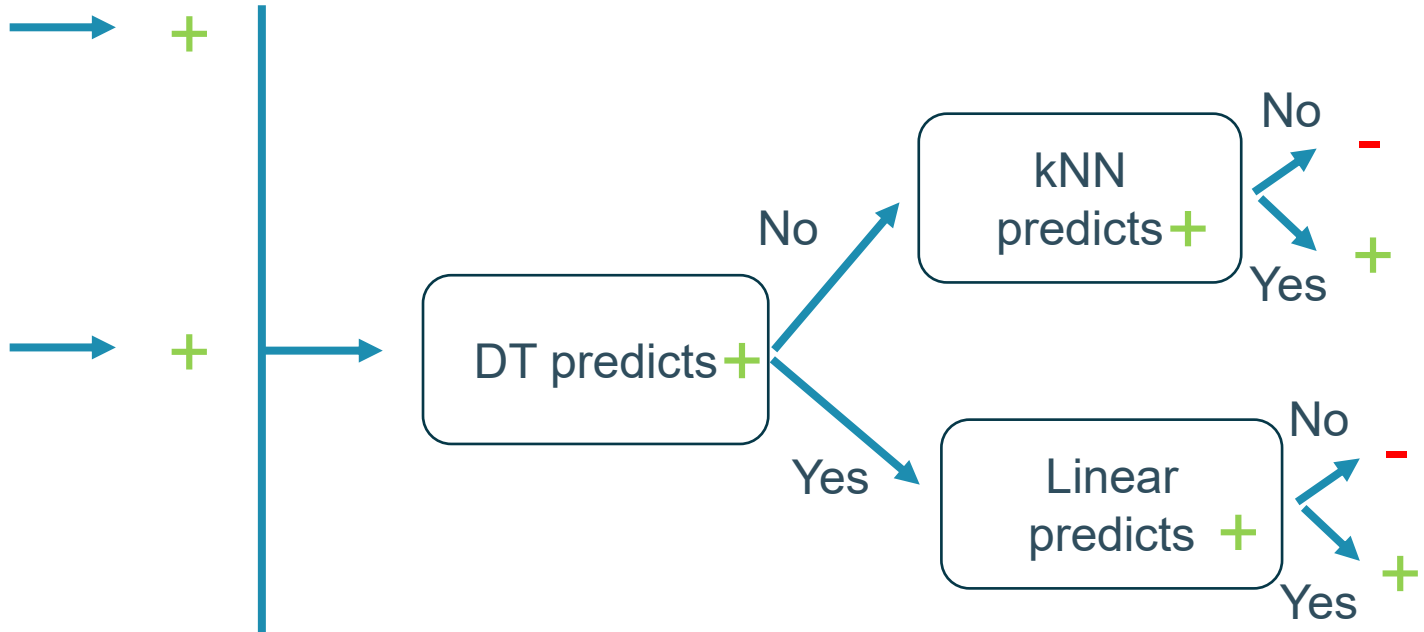


Stacking example on music



Stacking example on music

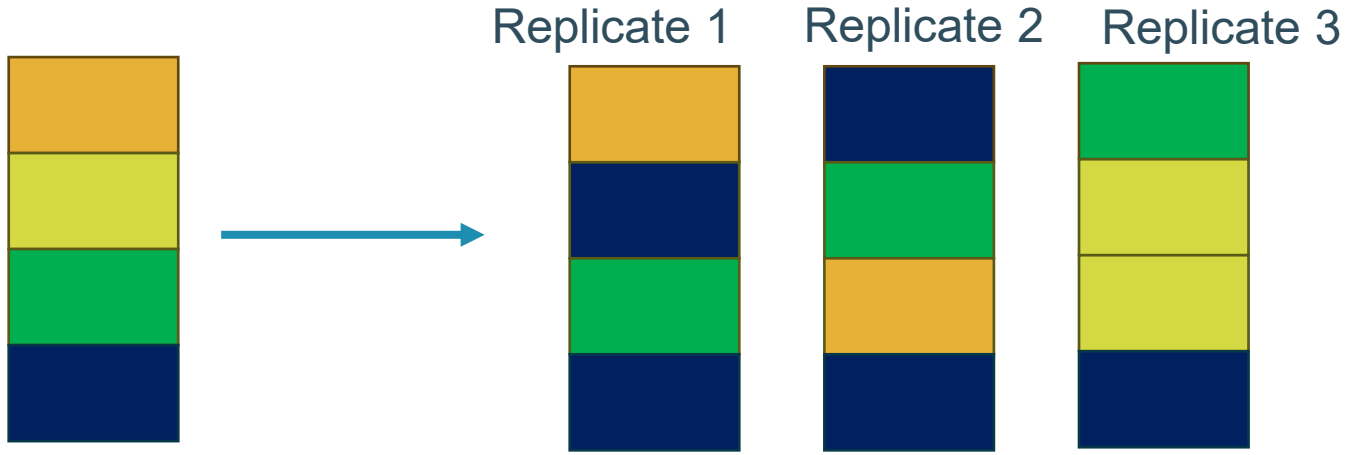




Use the same algorithm, vary the training data

Bootstrap

- Goal: multiple models that make different mistakes
- Problem: only one training set S
- Bootstrap replicates:
  - Construct bootstrap replicates to generate S' by sampling |S| examples with replacement from S



63.2% chance that an example appears in a replicate

Bootstrap Replicate

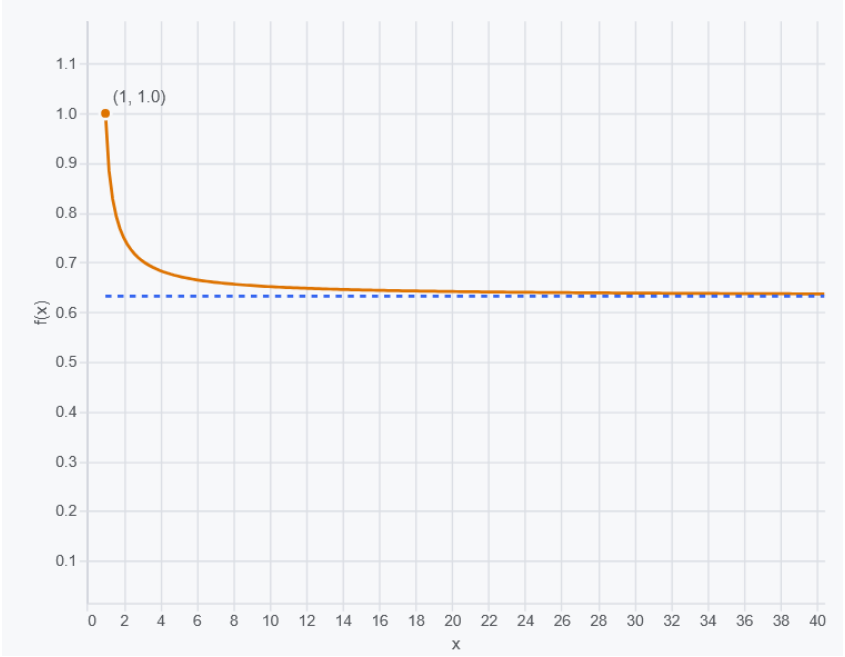
- **Claim**: 63.2% chance example x is in a bootstrap replicate S' with size |S|=n
- **Proof:**

$$P(x = S'_i) = \frac{1}{n} \quad \textit{x is sampled for i th position in S'}$$

$$P(x \neq S'_i) = 1 - \frac{1}{n}$$

$$P(x \notin S') = \left(1 - \frac{1}{n}\right)^n$$

$$P(x \in S') = 1 - \left(1 - \frac{1}{n}\right)^n \approx 1 - \frac{1}{e} \approx 0.632$$



Bagging: Bootstrap Aggregating

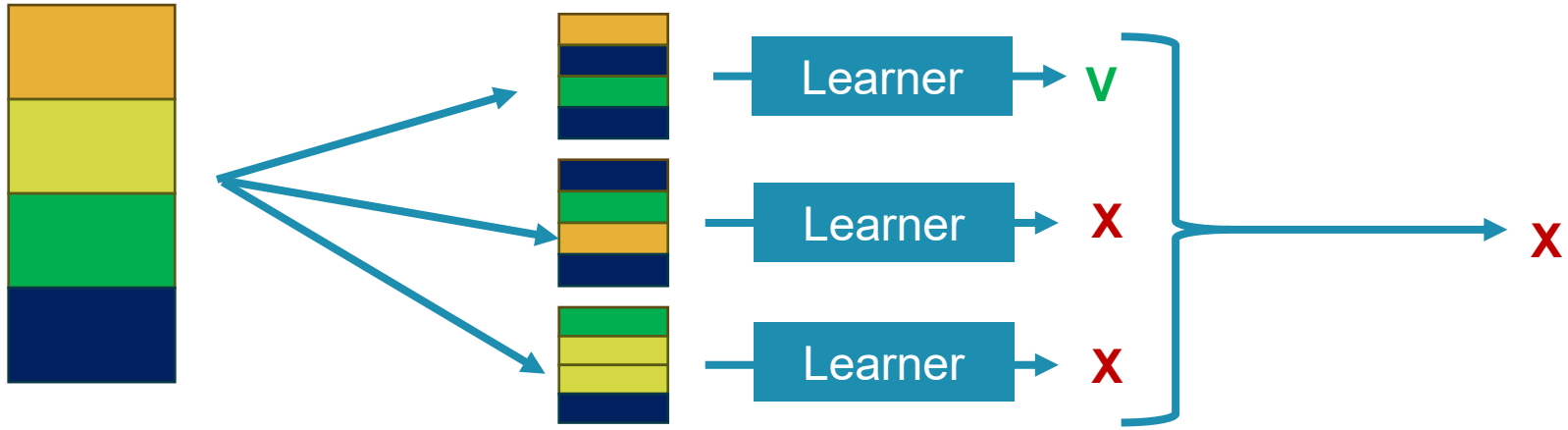
– **Given**: Data set S, integer T

For t = 1..T

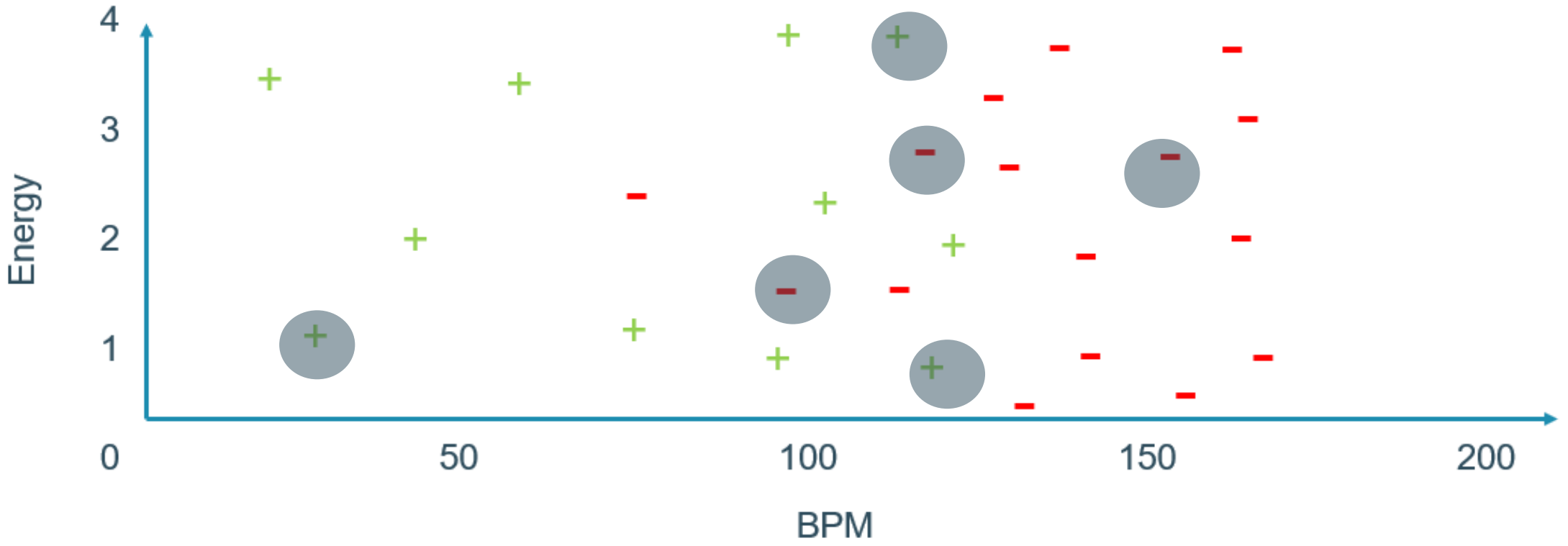
S_t=Bootstrap replicate of S

h_t= Apply learning algorithm to S_t

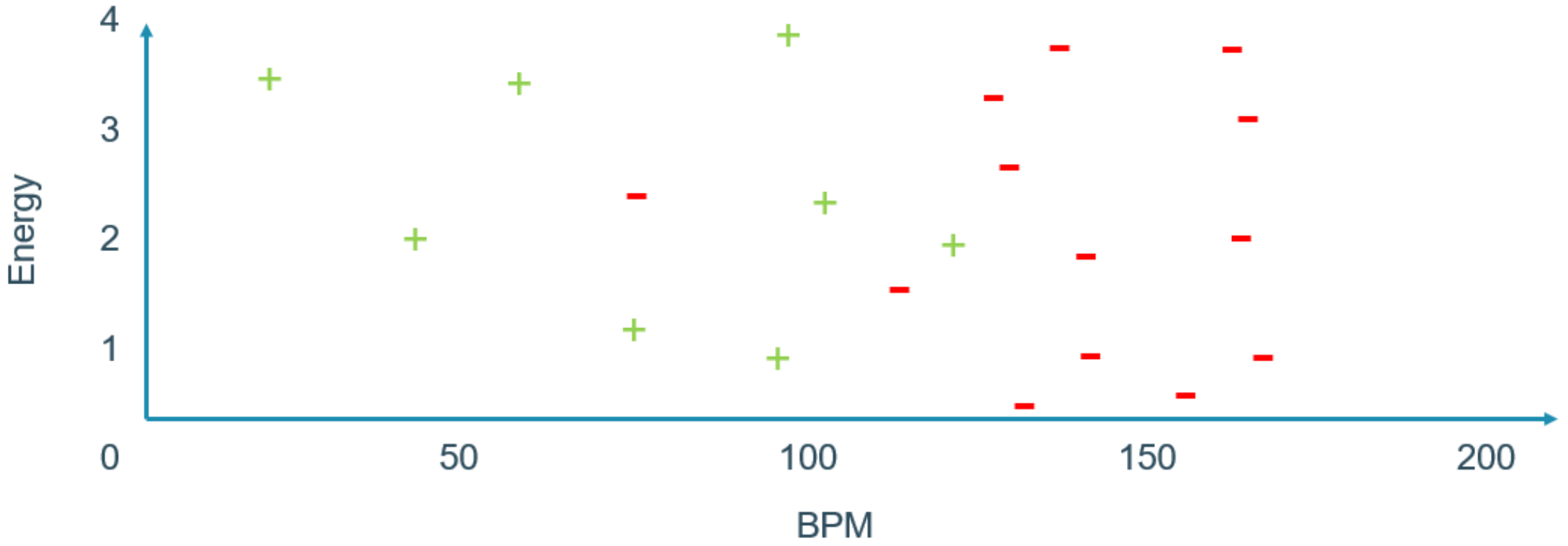
Combine all h_t using an unweighted vote



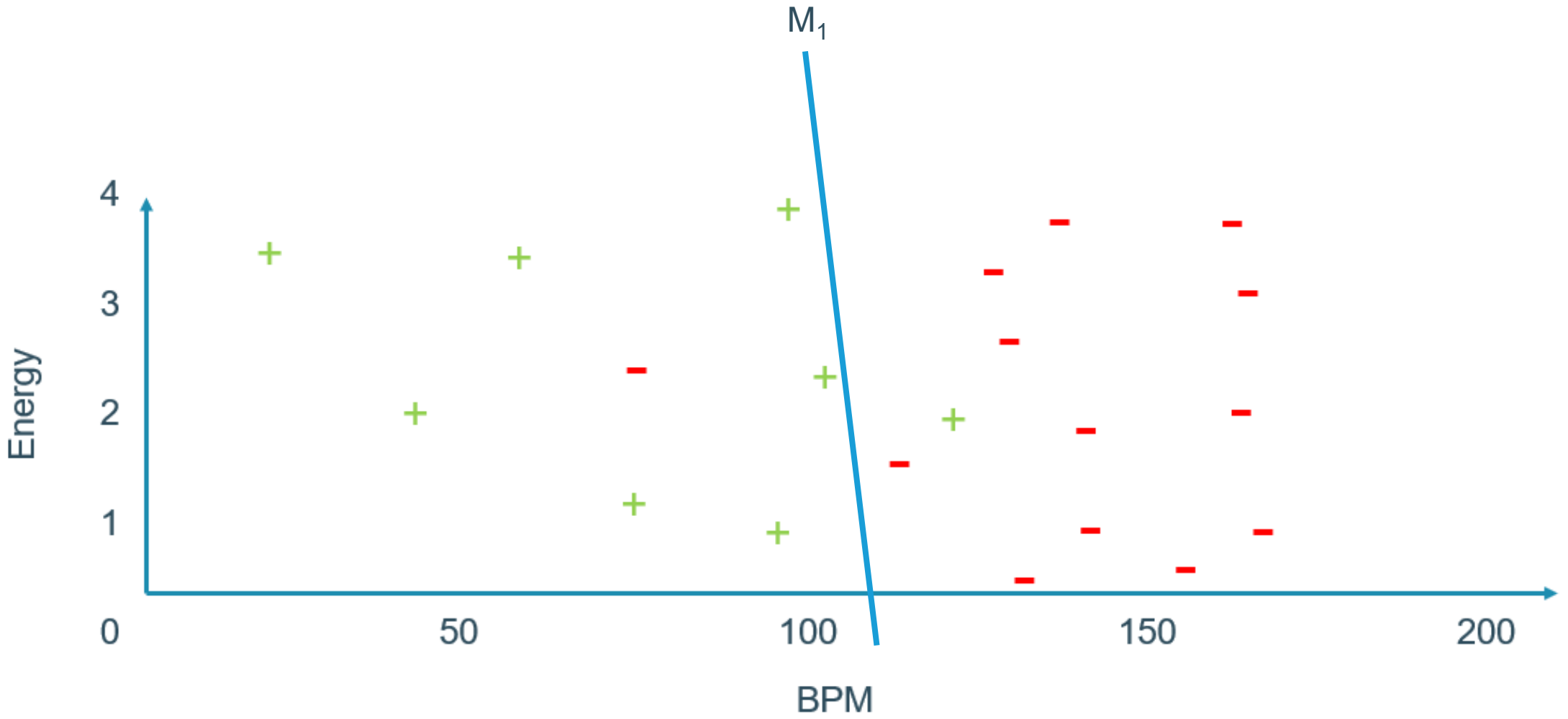
Bagging example on music



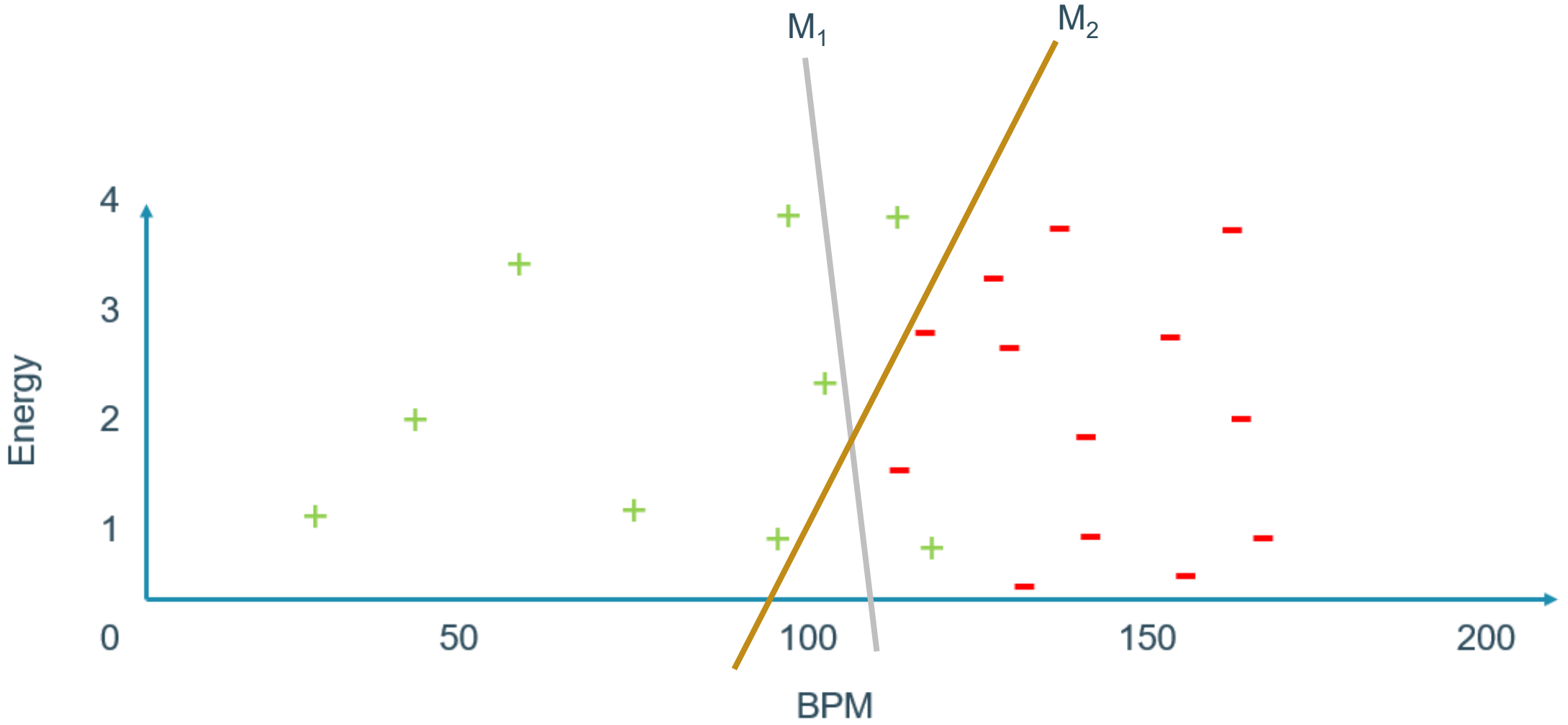
Bagging example on music



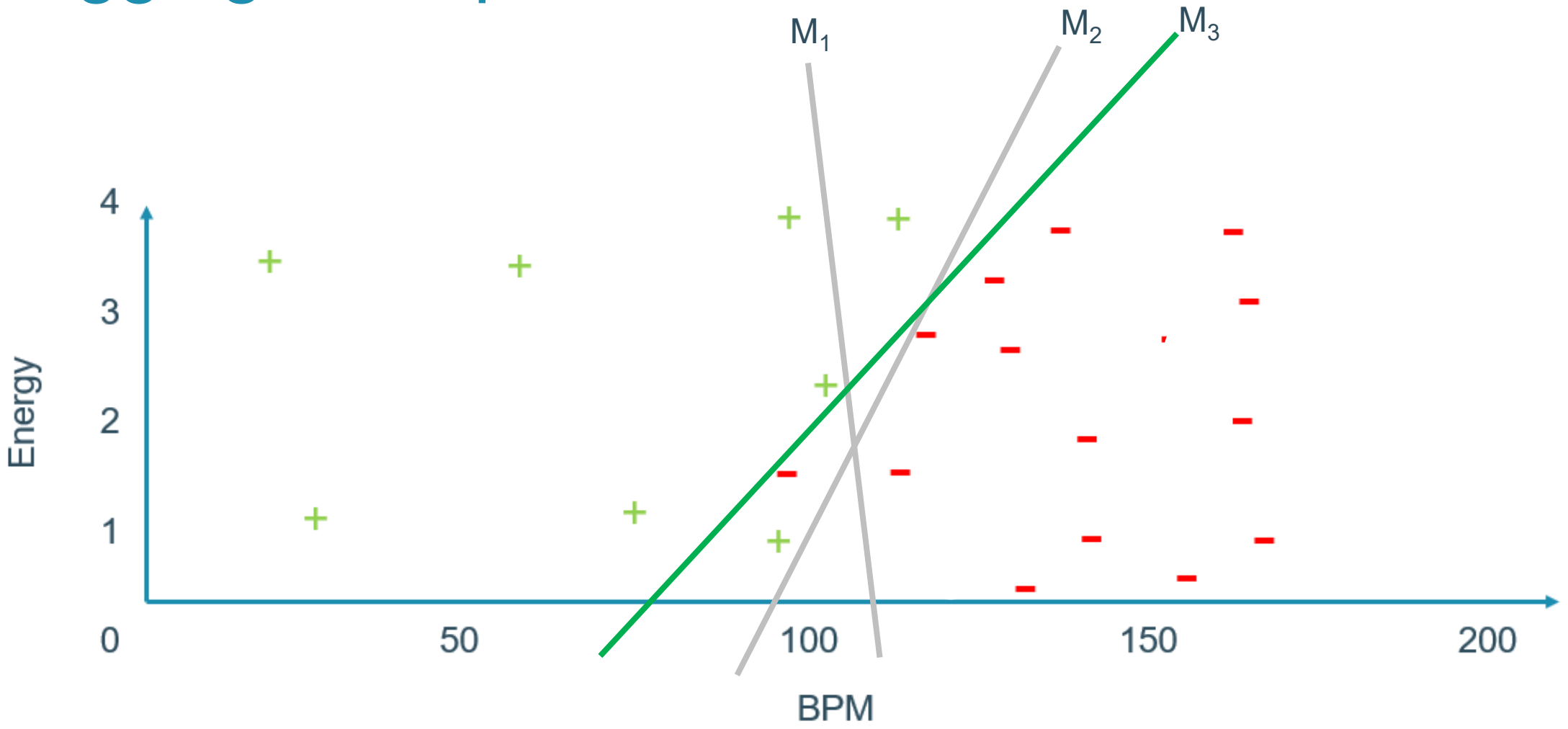
Bagging example on music



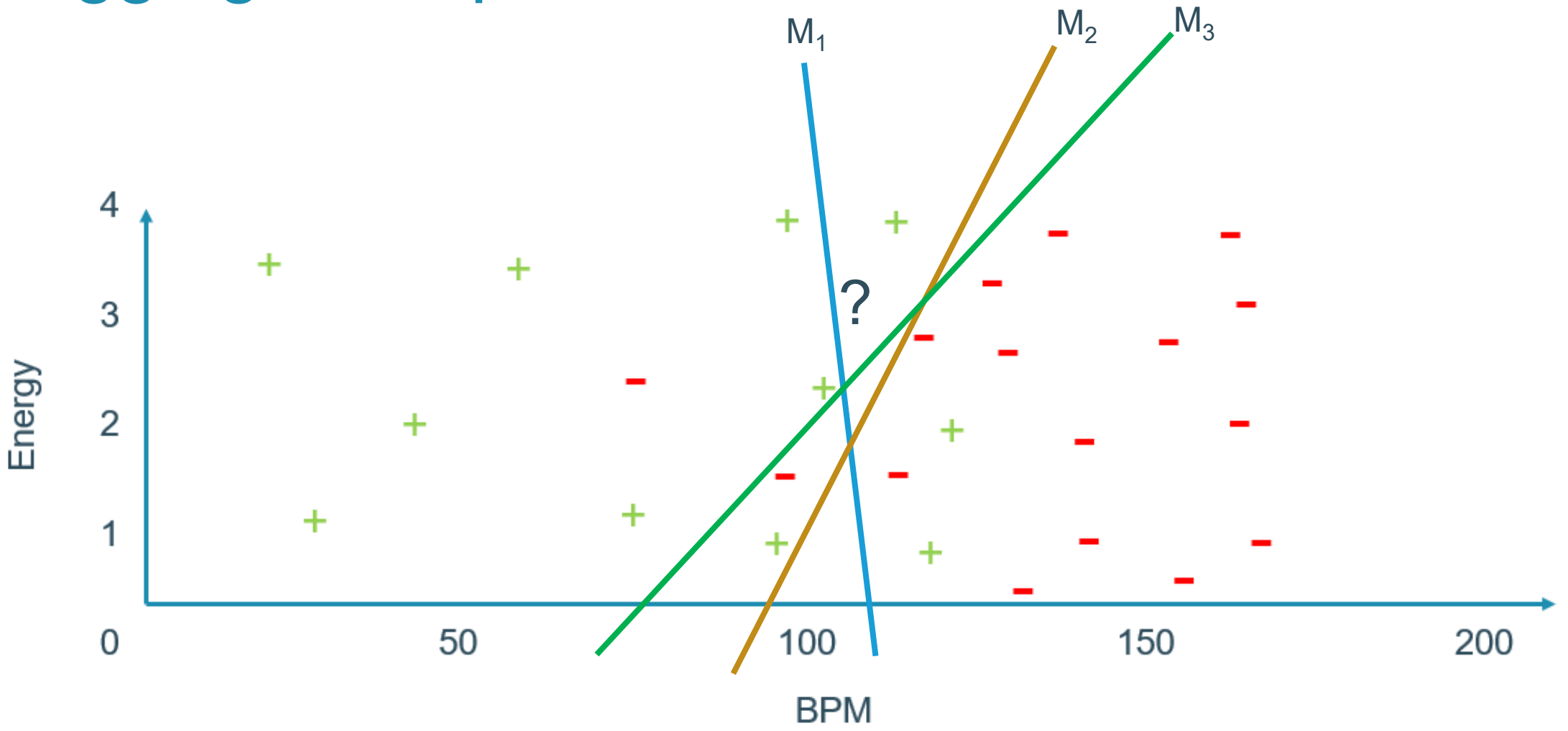
Bagging example on music



Bagging example on music



Bagging example on music

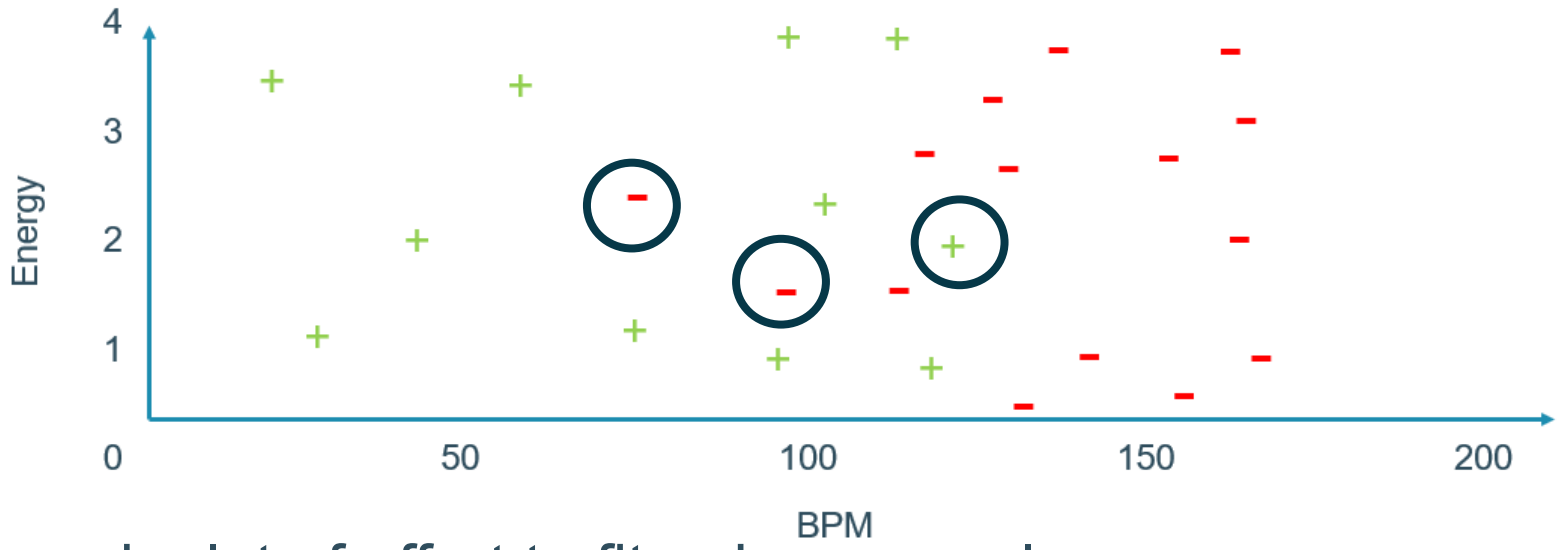


Sampling based ensembles

- Bagging works with any learner in principle
- Bagging works best for unstable learners
- Learner is UNSTABLE if: minor variations in training data results in major changes in classifier output
  - Unstable: Decision trees, neural networks, rule learning algorithms, 1NN
  - Stable: Linear regression, kNN (k>1), linear threshold algorithms

Intuition: why does bagging work?

– Suppose your data looks like this:



– Learners spend a lot of effort to fit noisy examples

– Noisy examples not in sample = better model

➔ Bootstrap replicates can generate “easier/better” data for the learner

Further decorrelate models: Random Forests

- A variant of Bagging
- **Given** Dataset S, integer T

  For i=1..T

    S_t= bootstrap replicate of S

    Build decision tree on S_t, but in each recursive call

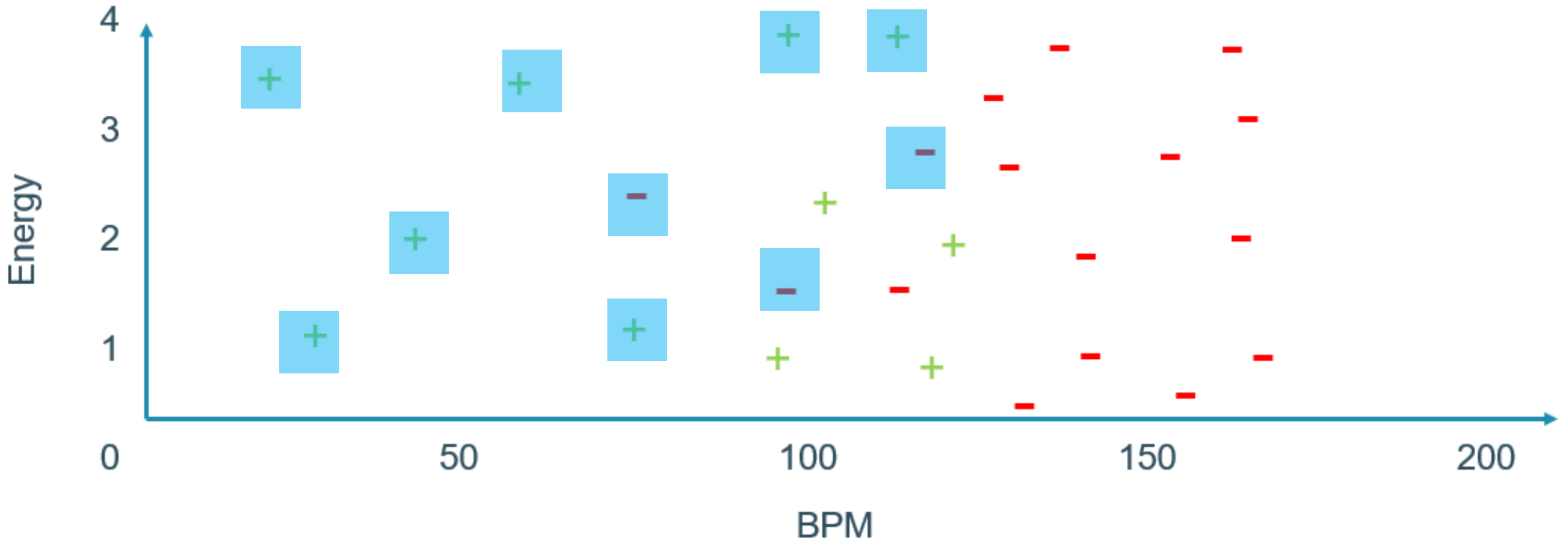
      Choose n features (without replacement)

      Choose best of these n as the root of this (sub)tree

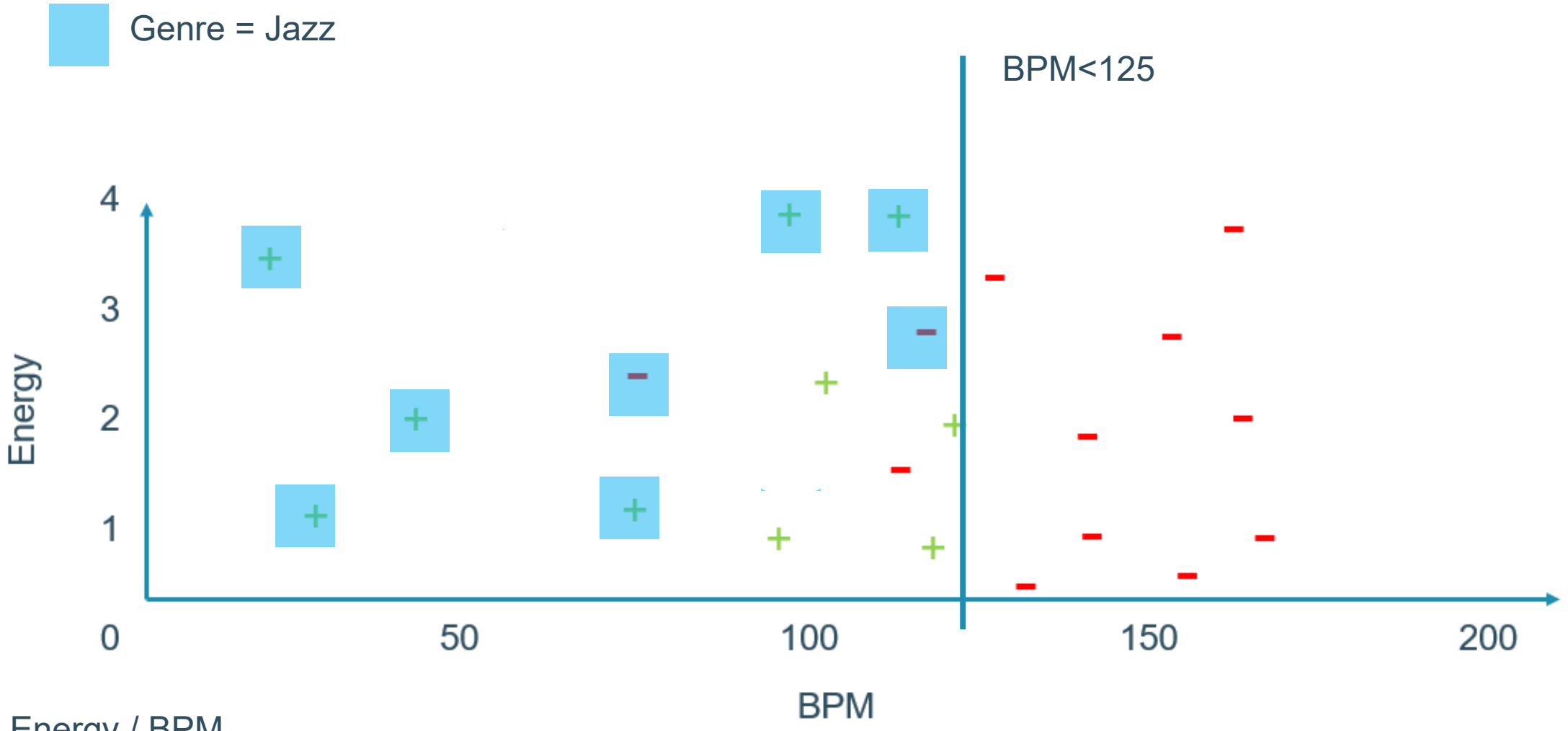
    Do not prune the tree

Random Forest example on music

Genre = Jazz

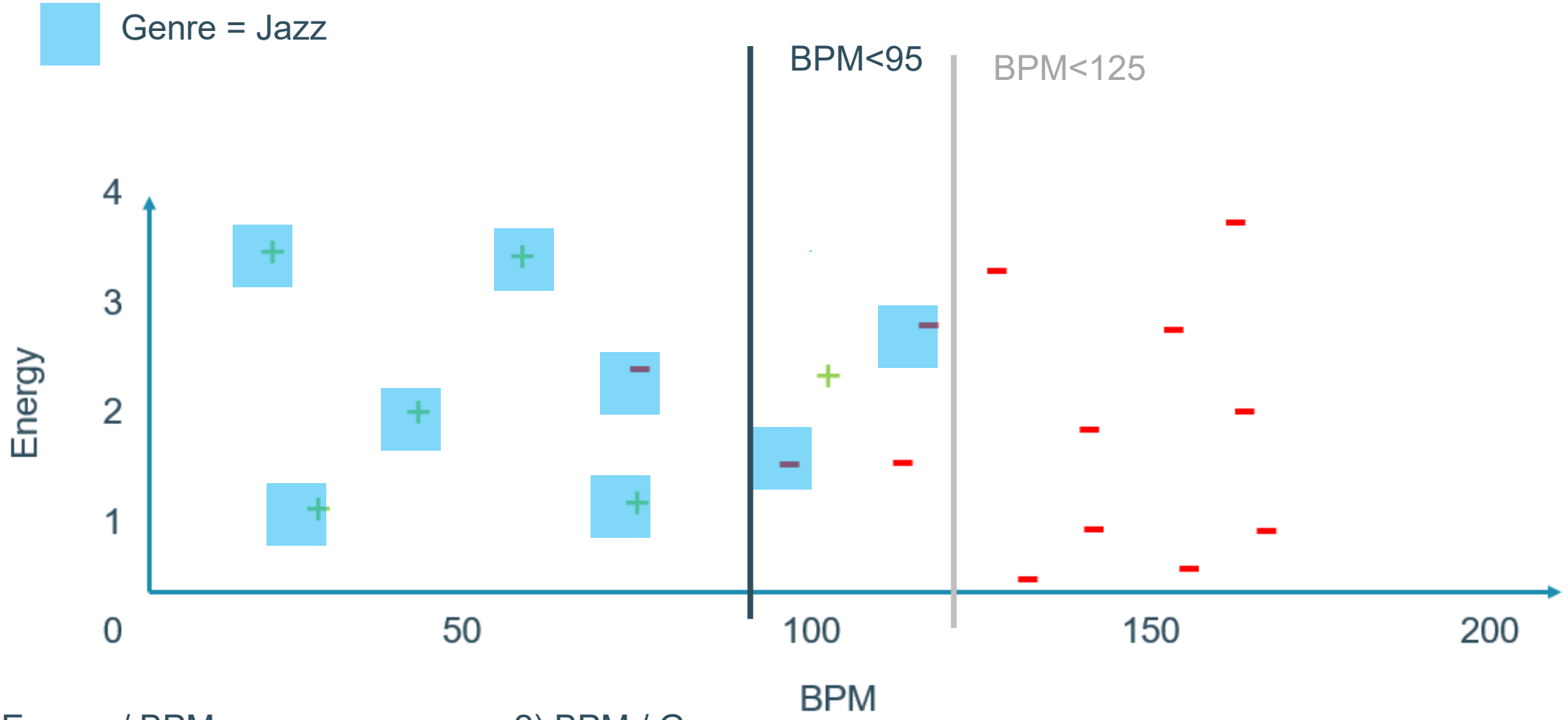


Random Forest example on music



1) Energy / BPM

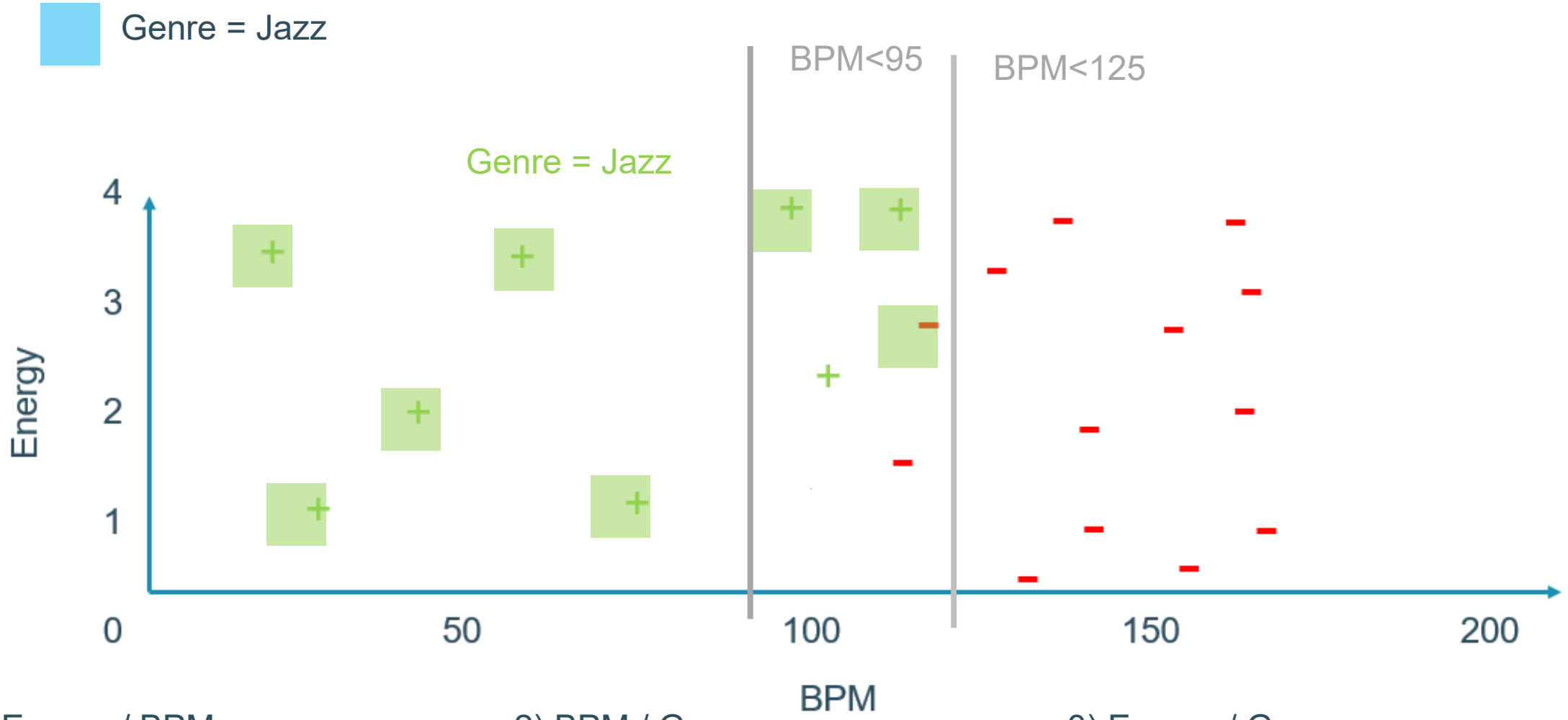
Random Forest example on music



1) Energy / BPM

2) BPM / Genre

Random Forest example on music

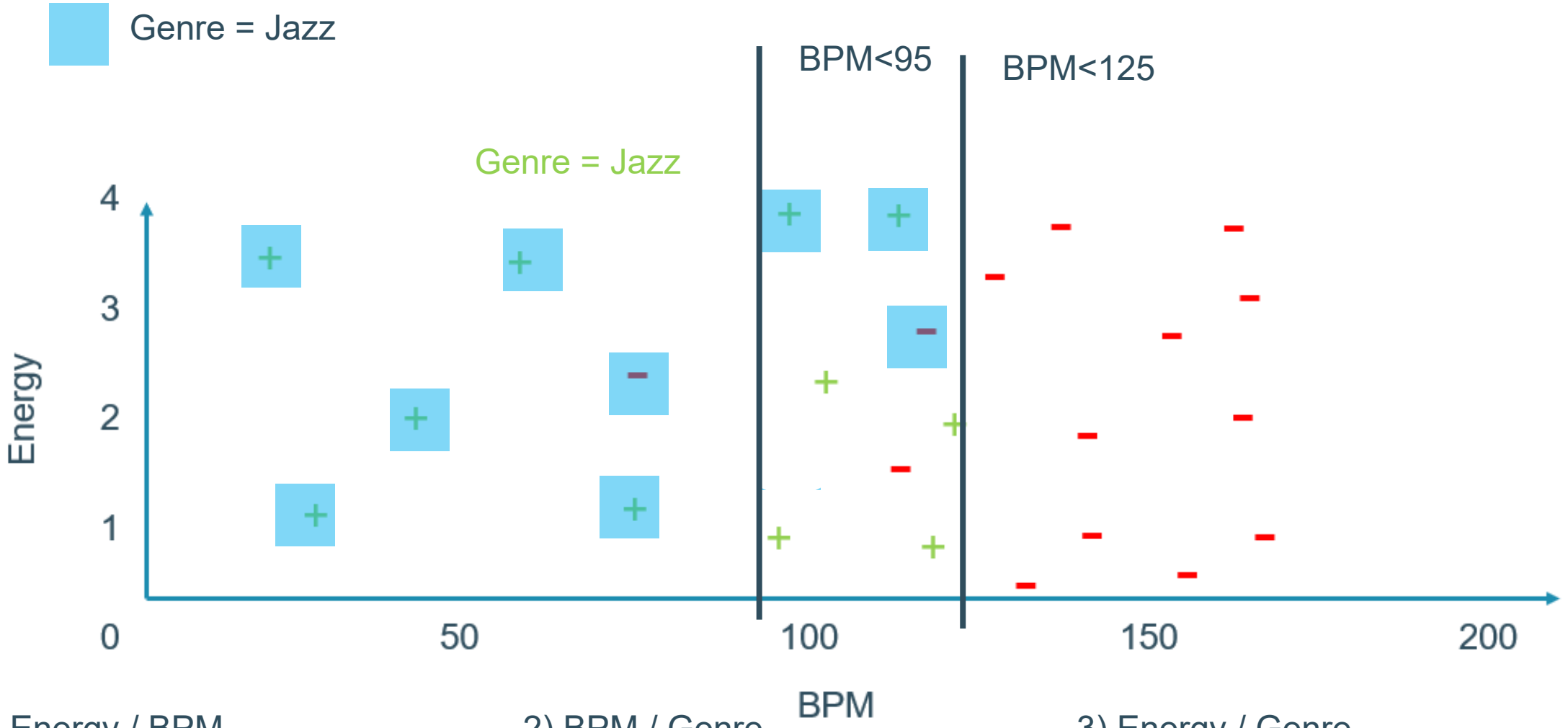


1) Energy / BPM

2) BPM / Genre

3) Energy / Genre

Random Forest example on music

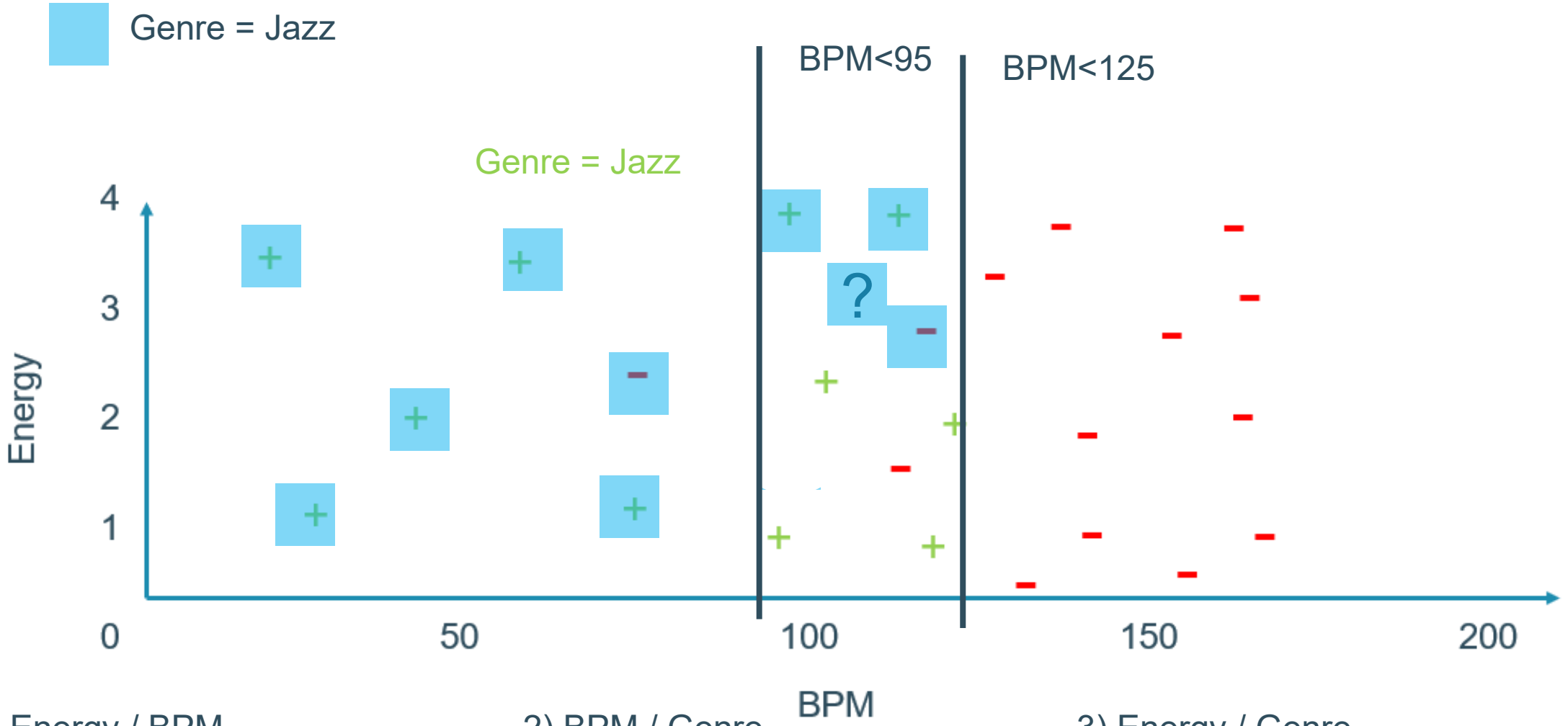


1) Energy / BPM

2) BPM / Genre

3) Energy / Genre

Random Forest example on music



1) Energy / BPM

2) BPM / Genre

3) Energy / Genre

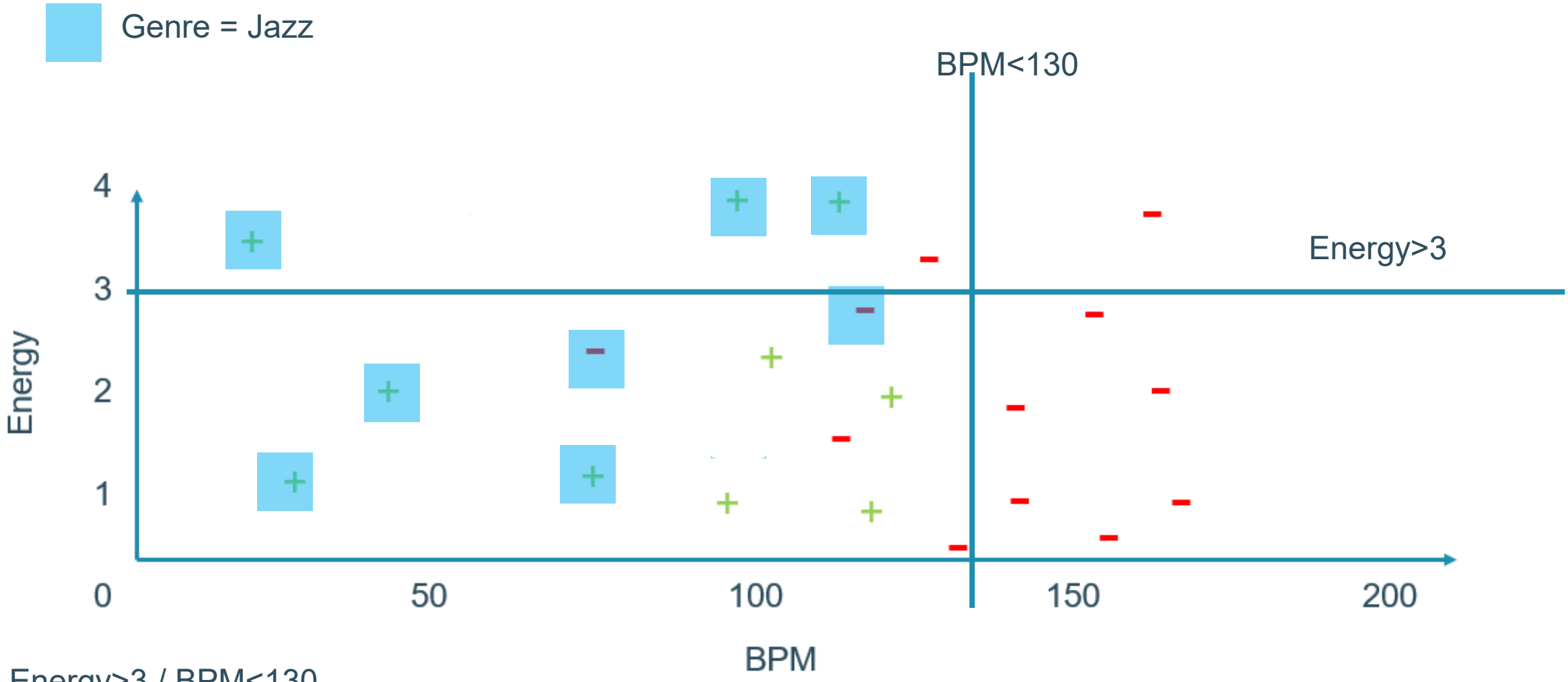
More on Random Forests

- Increasing n increases
  - Correlation among individual trees through shared features (BAD)
  - Accuracy of individual trees (GOOD)
- Using a tuning set to pick a good value for n
- Overall, random forests are
  - Very fast
  - Able to deal with large numbers of features
  - Reducing overfitting substantially
  - Working very well in practice

Other Idea: extremely random trees

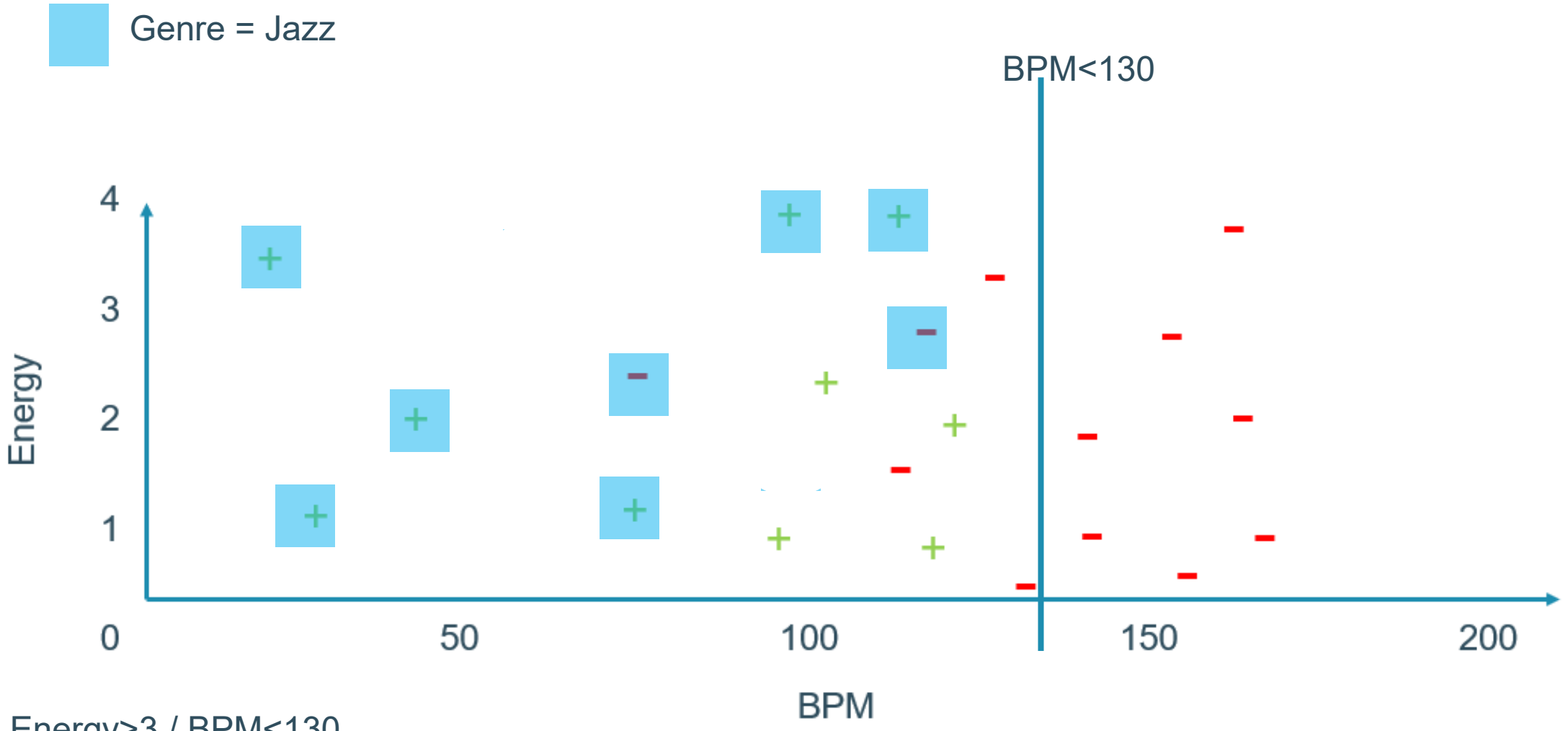
- **Given** Dataset S, integer T
  For i=1..T
    S_t= bootstrap replicate of S
    Build decision tree on S_t, but in each recursive call
      Choose n features (without replacement)
      Choose random split points for these features
      Choose best of these n as the root of this (sub)tree
    Do not prune the tree

Random Forest example on music



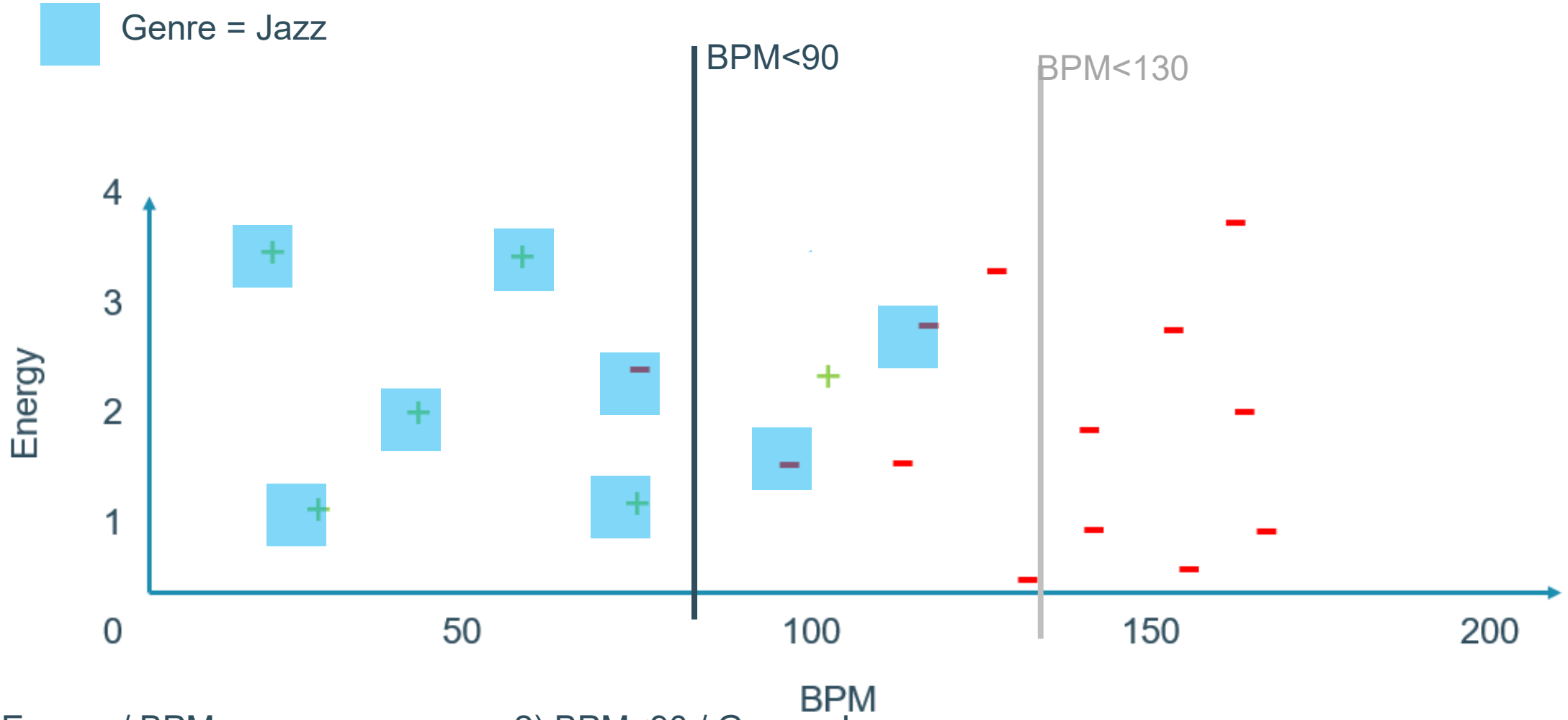
1) Energy>3 / BPM<130

Random Forest example on music



1) Energy>3 / BPM<130

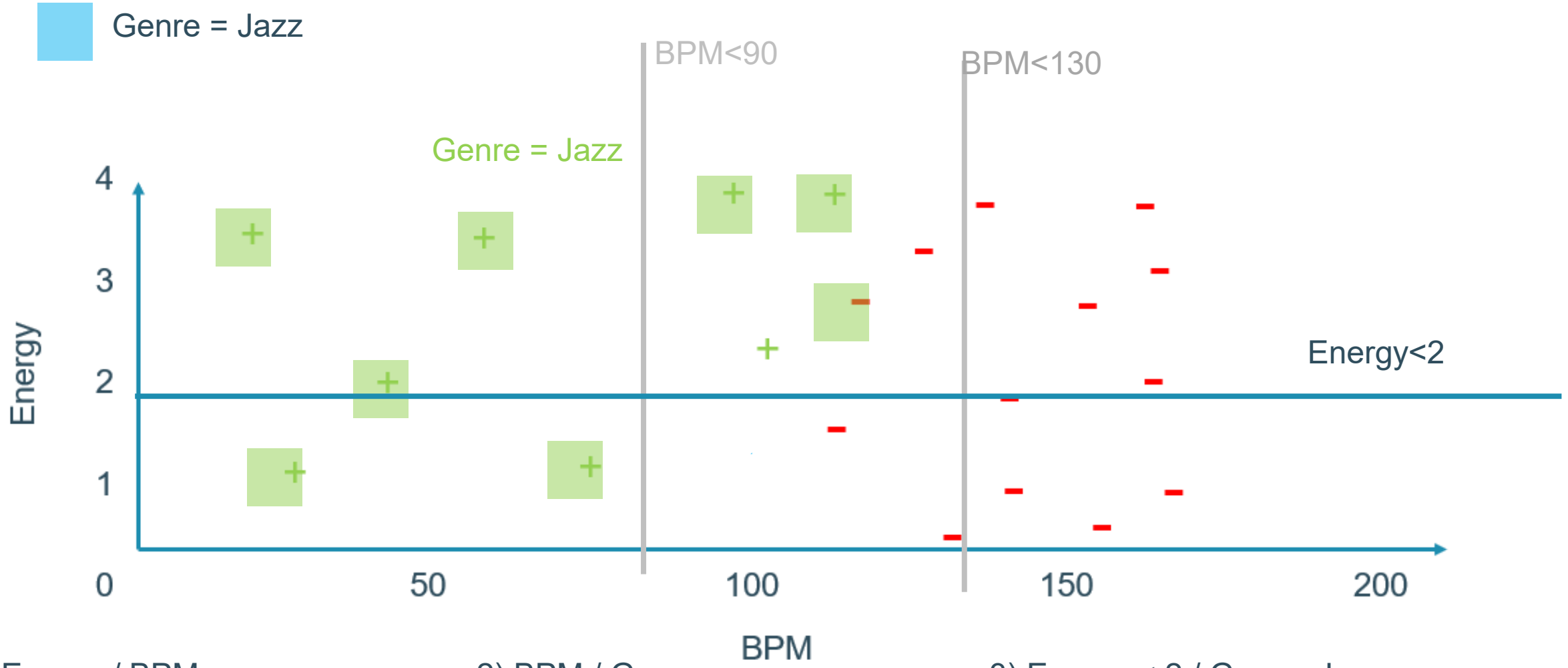
Random Forest example on music



1) Energy / BPM

2) BPM<90 / Genre=Jazz

Random Forest example on music

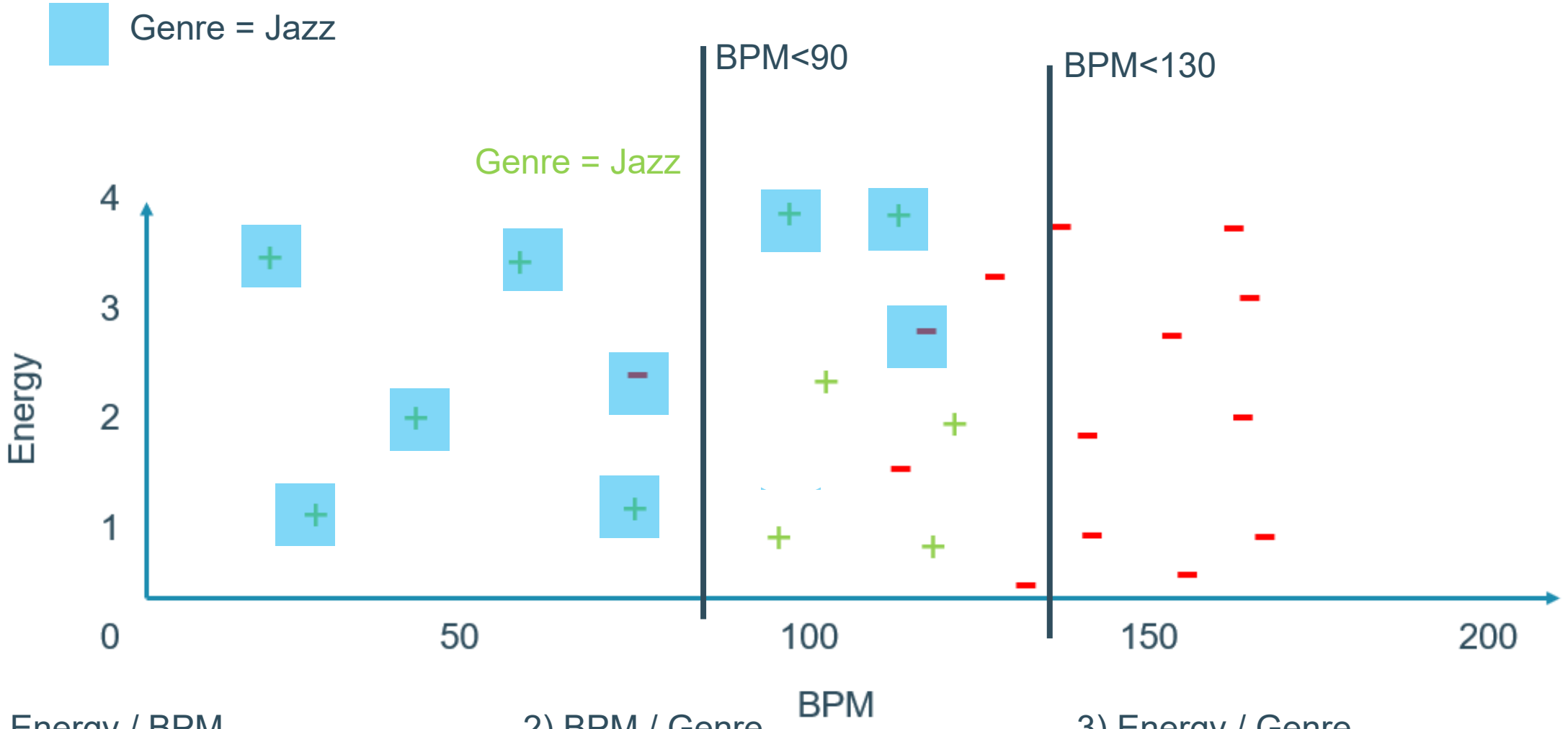


1) Energy / BPM

2) BPM / Genre

3) Energy < 2 / Genre=Jazz

Random Forest example on music



1) Energy / BPM

2) BPM / Genre

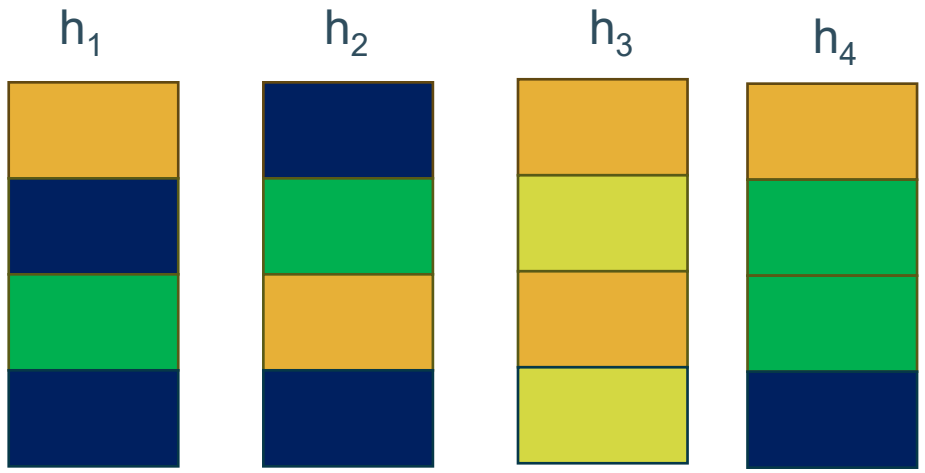
3) Energy / Genre

Alternative evaluation: Out of bag error

- How well does each learned model generalize?
- For each example *i* in the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on *i* and compare with label
  - Take the average of these errors

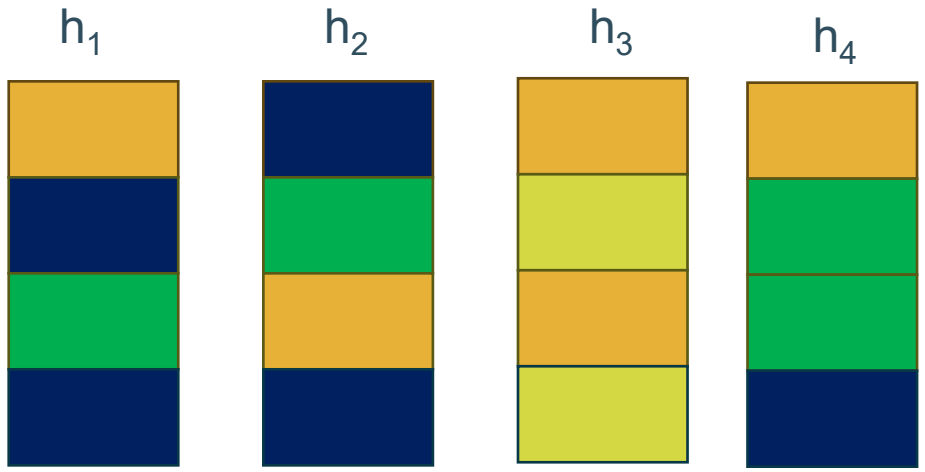
Alternative evaluation: Out of bag error

- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors



Alternative evaluation: Out of bag error

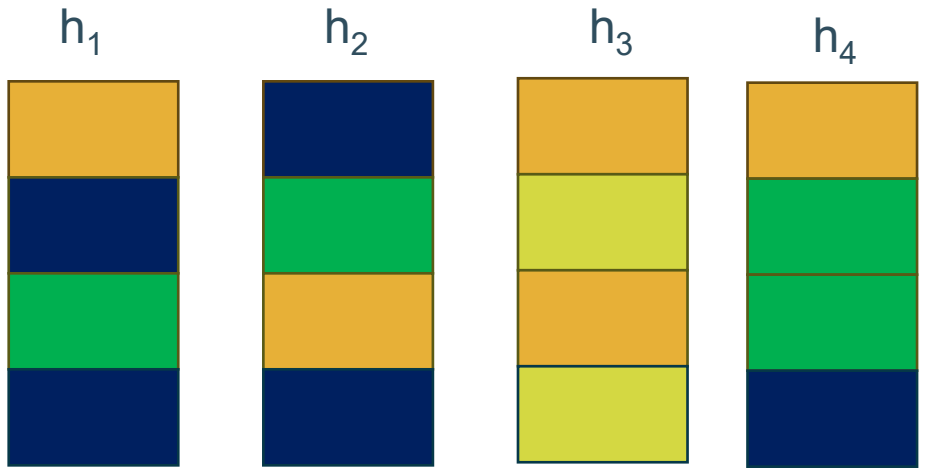
- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors

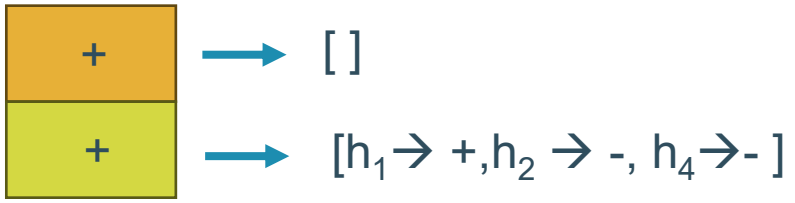




Alternative evaluation: Out of bag error

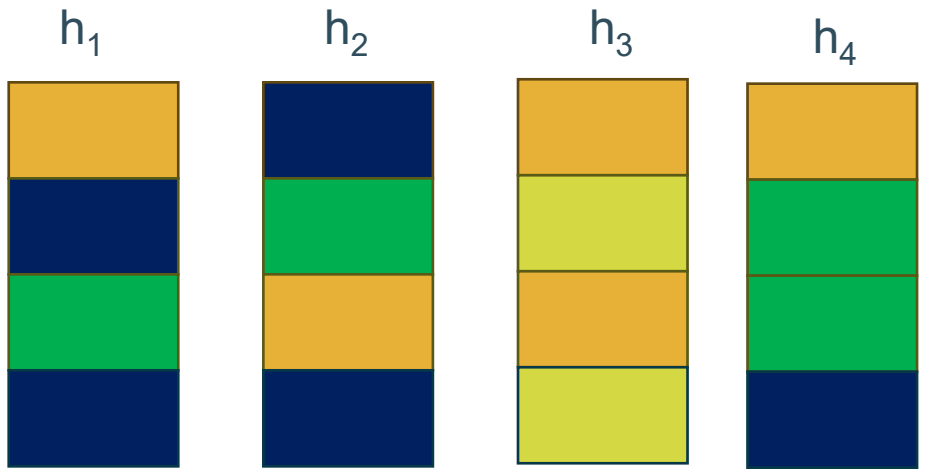
- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors

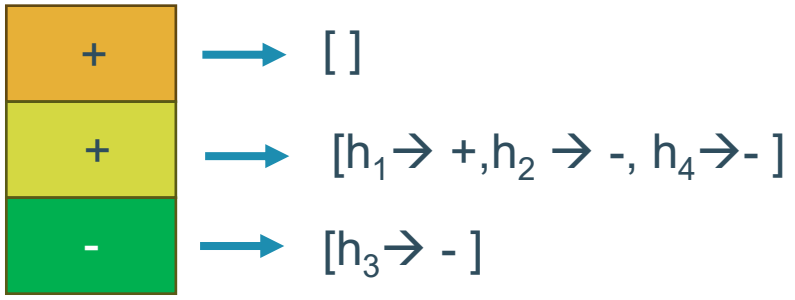




Alternative evaluation: Out of bag error

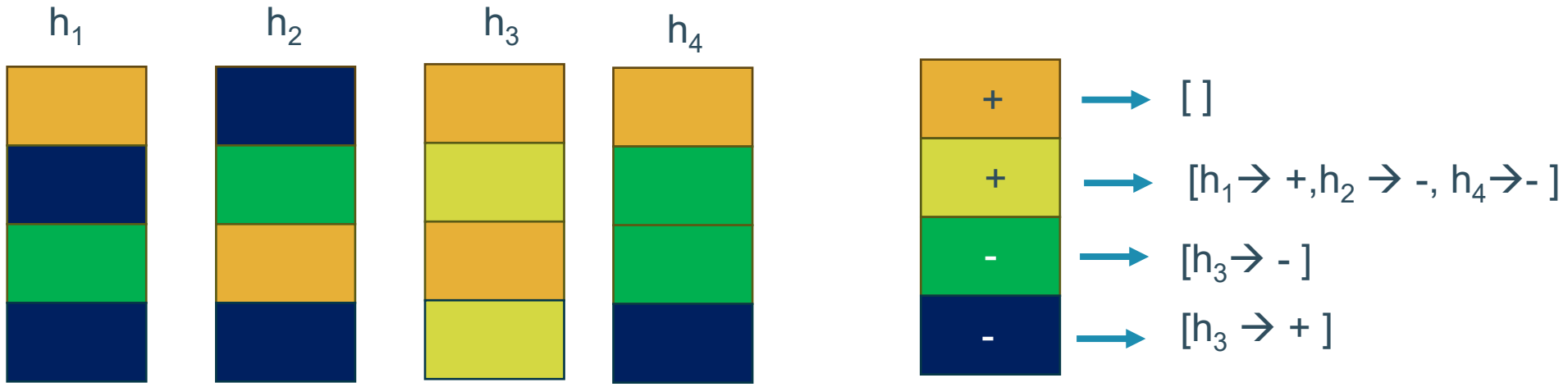
- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors





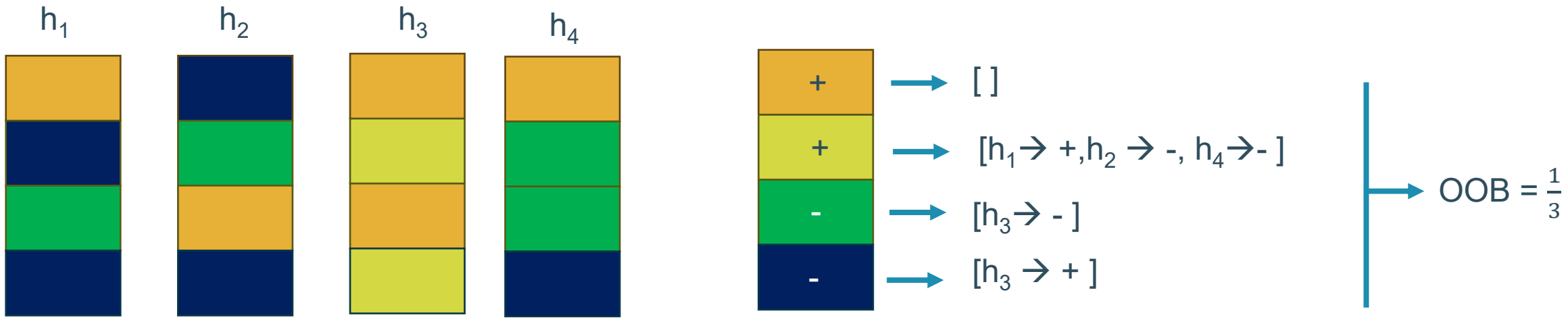
Alternative evaluation: Out of bag error

- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors



Alternative evaluation: Out of bag error

- How well does each learned model generalize?
- For each example i the database
  - Find all the models that did not train on this example
  - Take the majority vote of these models' predictions on i and compare with label
  - Take the average of these errors



Learn from previous mistakes

Boosting

- Typically, we want “strong learners”, i.e., high chance to get small error rate
- Boosting works with “weak learner”: Models are (slightly) better than random guessing
- General approach
  - Learn an additive model
  - Greedily add a model at a time
  - Current model focuses on correcting mispredictions from the previous models

Adaboost

- Works for binary classification problems
- Learns additive model iteratively

$$F(X) = \alpha_1 h_1(\mathrm{X}) + \alpha_2 h_2(X) + \cdots + \alpha_t h_t(X)$$

- Two big ideas when adding new model h_{t+1}:
- Idea 1: Weight examples that are classified incorrectly by h_t
- Idea 2: Prediction is a weighted vote where more accurate models have a higher weight α

AdaBoost

Given: Training examples *E*, Integer *T*

Give every example $e \in E$ equal weight $w_e = \frac{1}{|E|}$

For t = 1 … T:

Train a simple classifier h_t using the current example weights

$\varepsilon_t \leftarrow \sum_{\substack{e \in E \\ h_t(e) \neq y_e}} w_e$ *Sum the weights of all incorrectly classified examples given labels* y_e

$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$

For every training example $e \in E$:

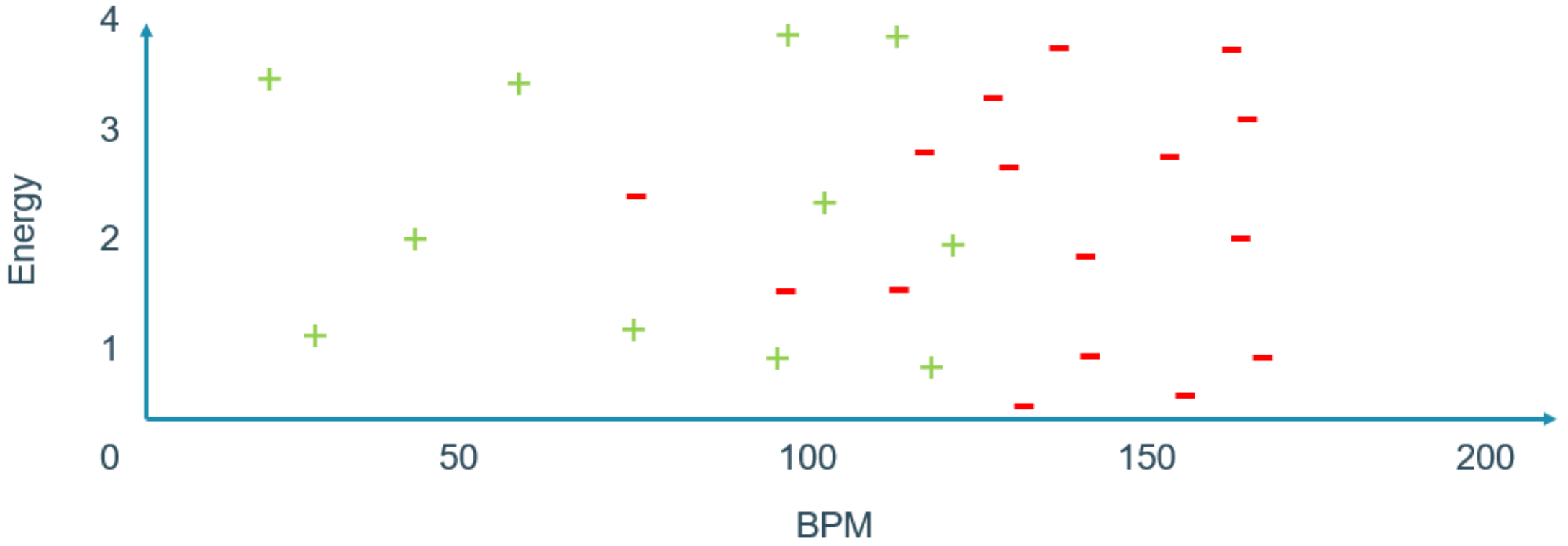
If h_t classifies it incorrectly → Increase its weight w_e

Otherwise → Decrease its weight w_e

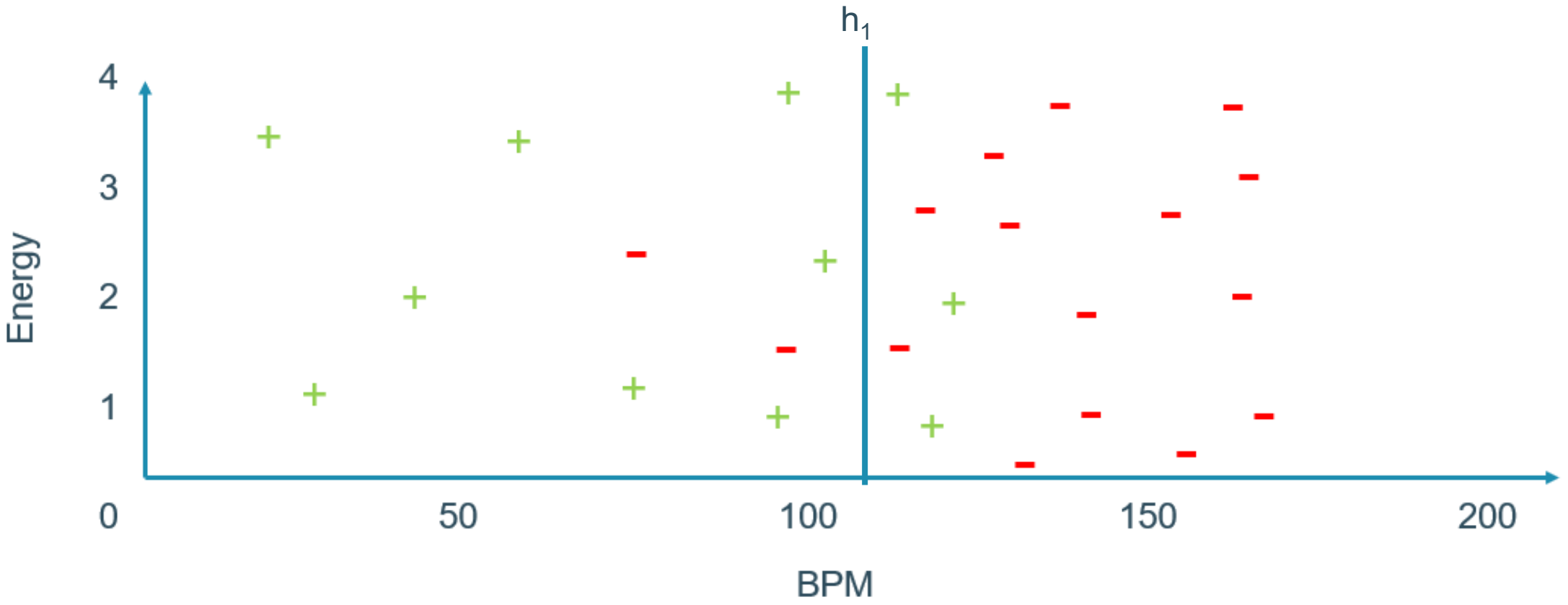
Normalize the example weights to 1: $w_e = \frac{w_e}{\Sigma_{e \in E} w_e}$

Add h_t with voting strength α_t to the ensemble

AdaBoost example music

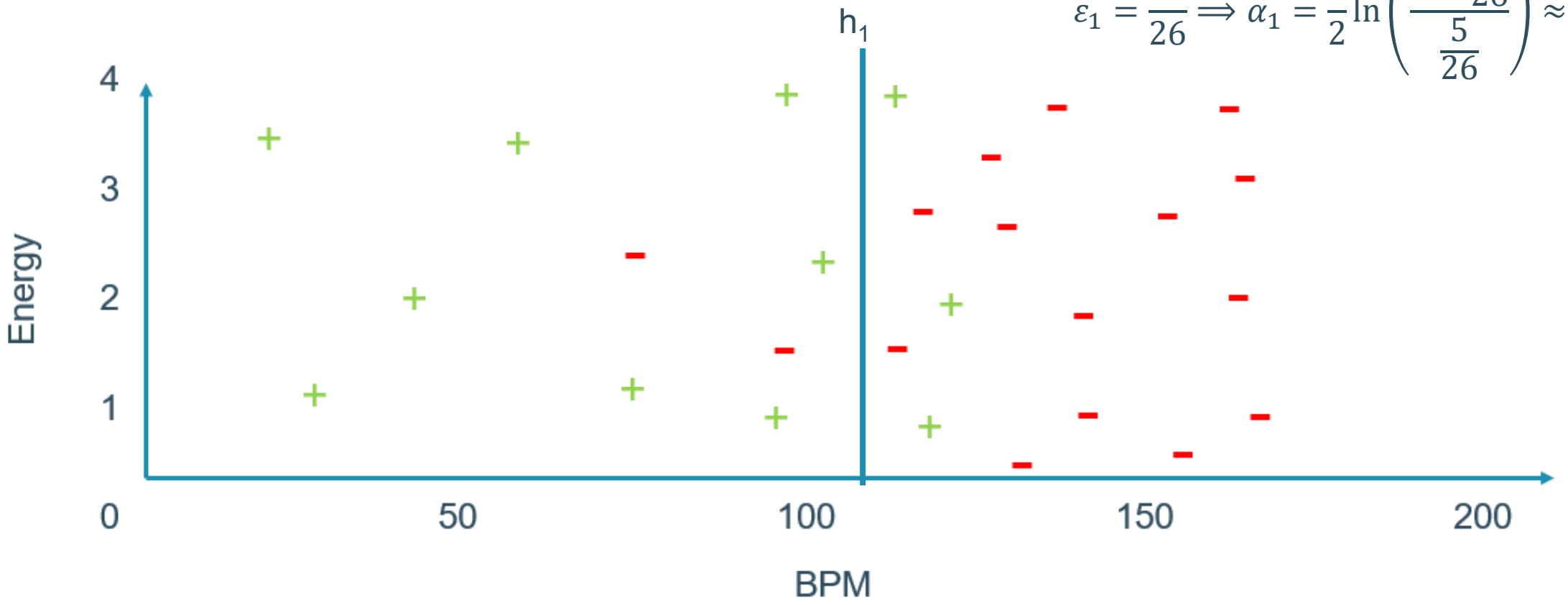


AdaBoost example music



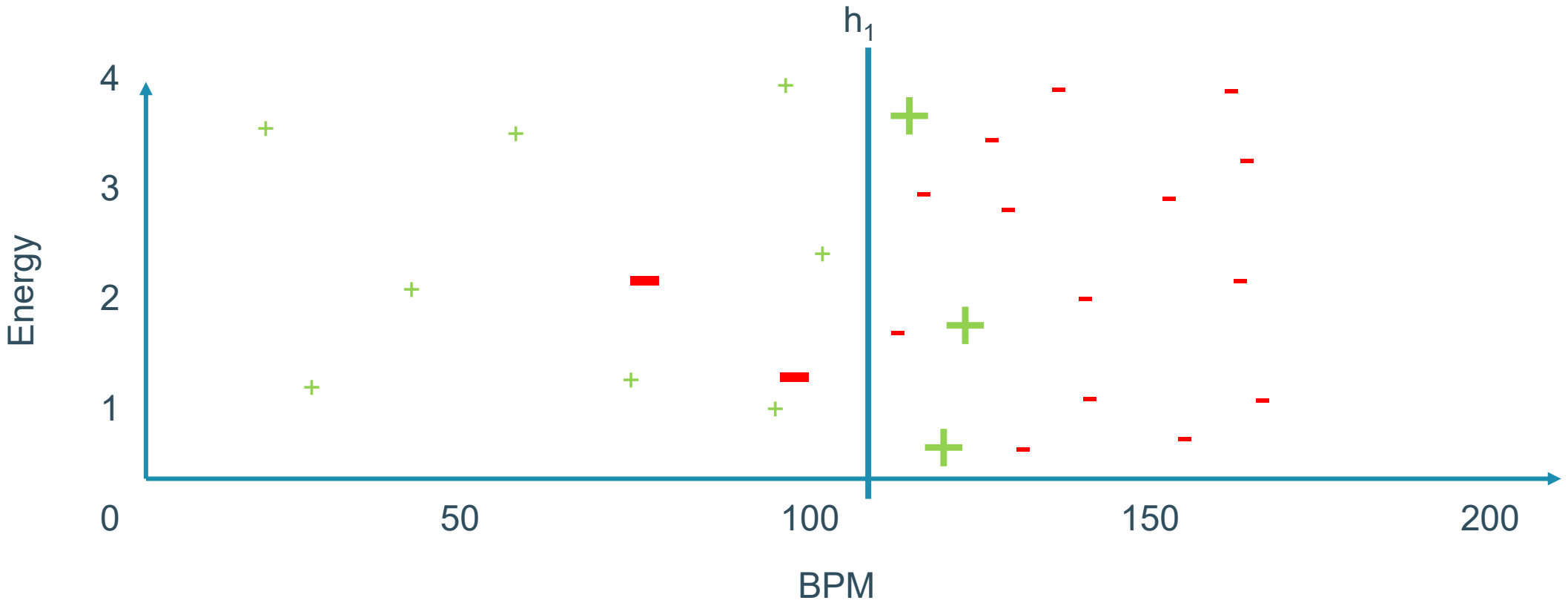
AdaBoost example music

$$\varepsilon_1 = \frac{5}{26} \Rightarrow \alpha_1 = \frac{1}{2}\ln\left(\frac{1-\frac{5}{26}}{\frac{5}{26}}\right) \approx 0.72$$



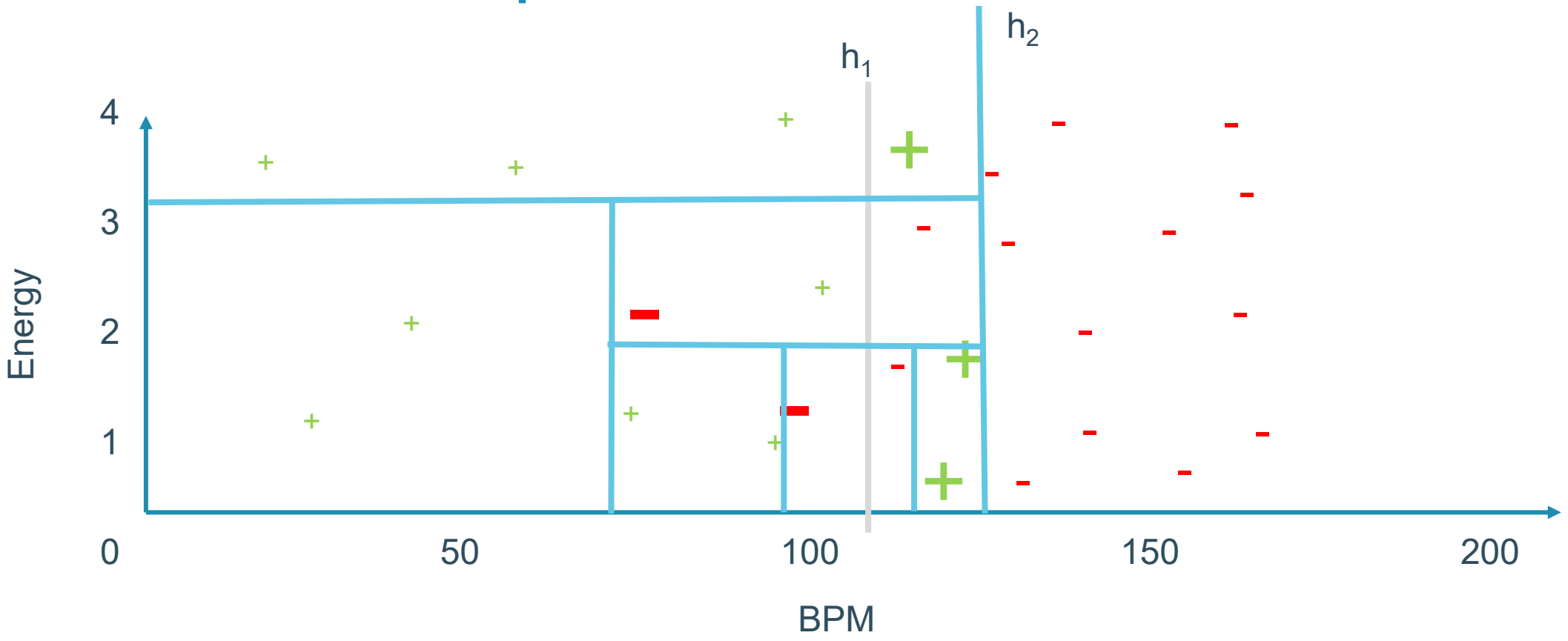
F(x) = 0.72 $h_1(x)$

AdaBoost example music



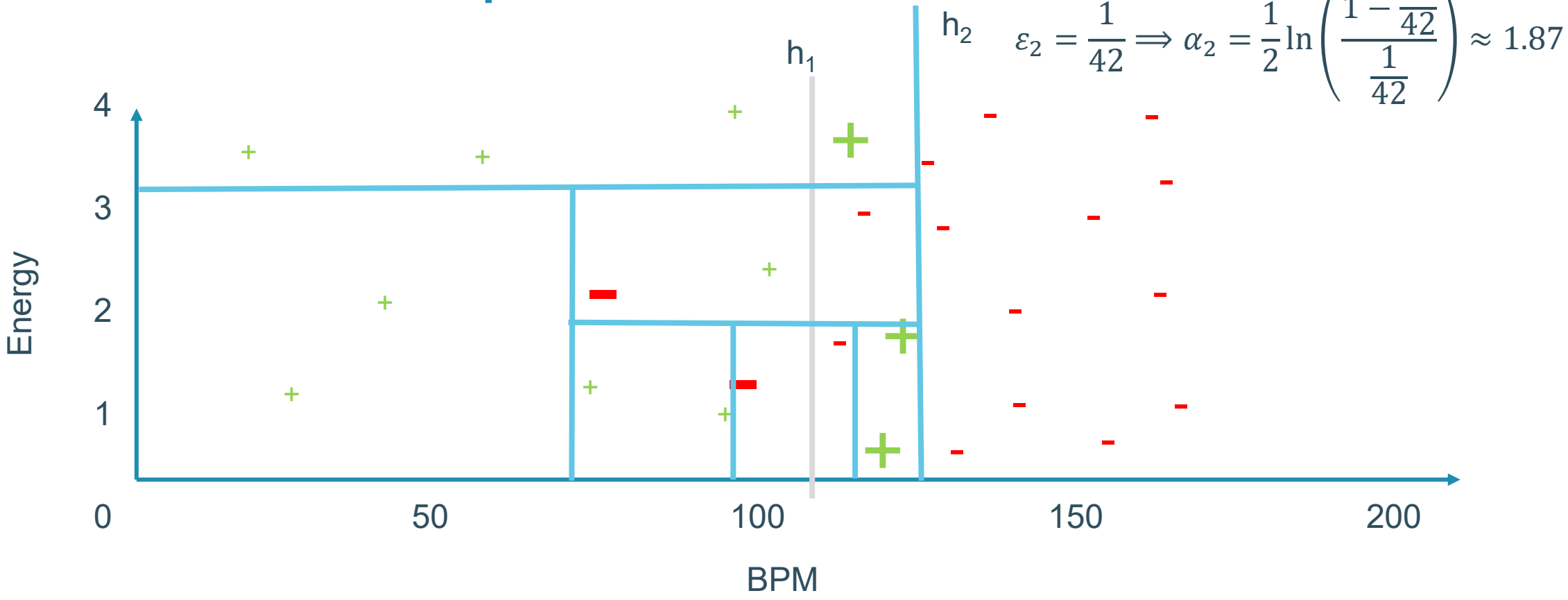
$F(x) = 0.72\ h_1(x)$

AdaBoost example music



$F(x) = 0.72\ h_1(x)$

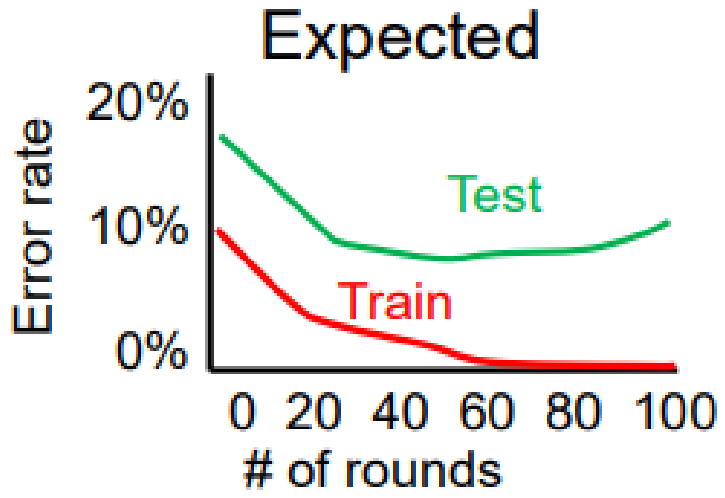
AdaBoost example music

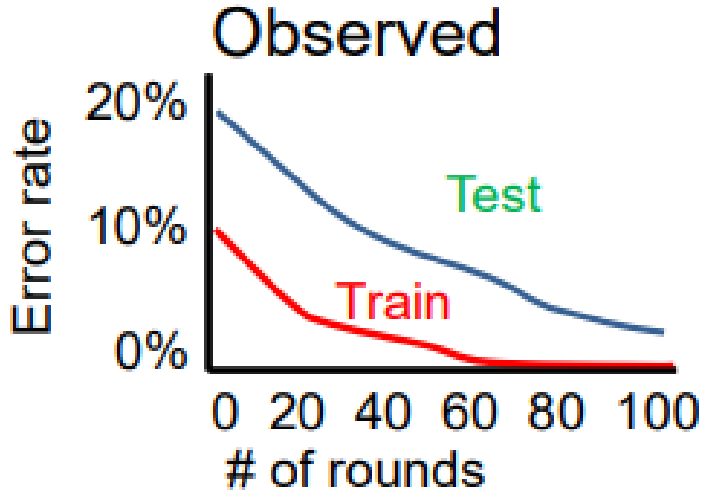


$F(x) = 0.72\, h_1(x) + 1.87\, h_2(x)$

How will the number of rounds affect generalization?

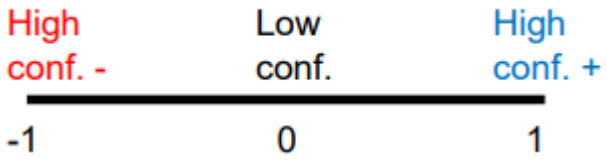
- Expect: Training error drops to 0, but test error increases
- Observed: Even after 1000 rounds, test error is stable and continues to drop even after training error is 0





Margins

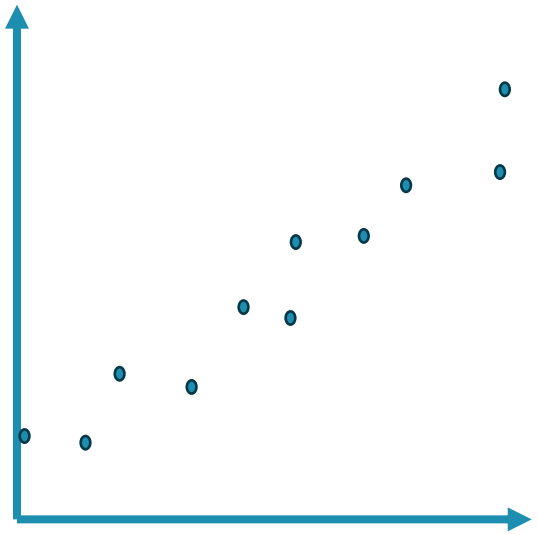
- **Key idea:** Error only measures if classification is correct, but confidence of prediction is important
- Measure confidence by margin: (weighted vote +) – (weighted vote -)

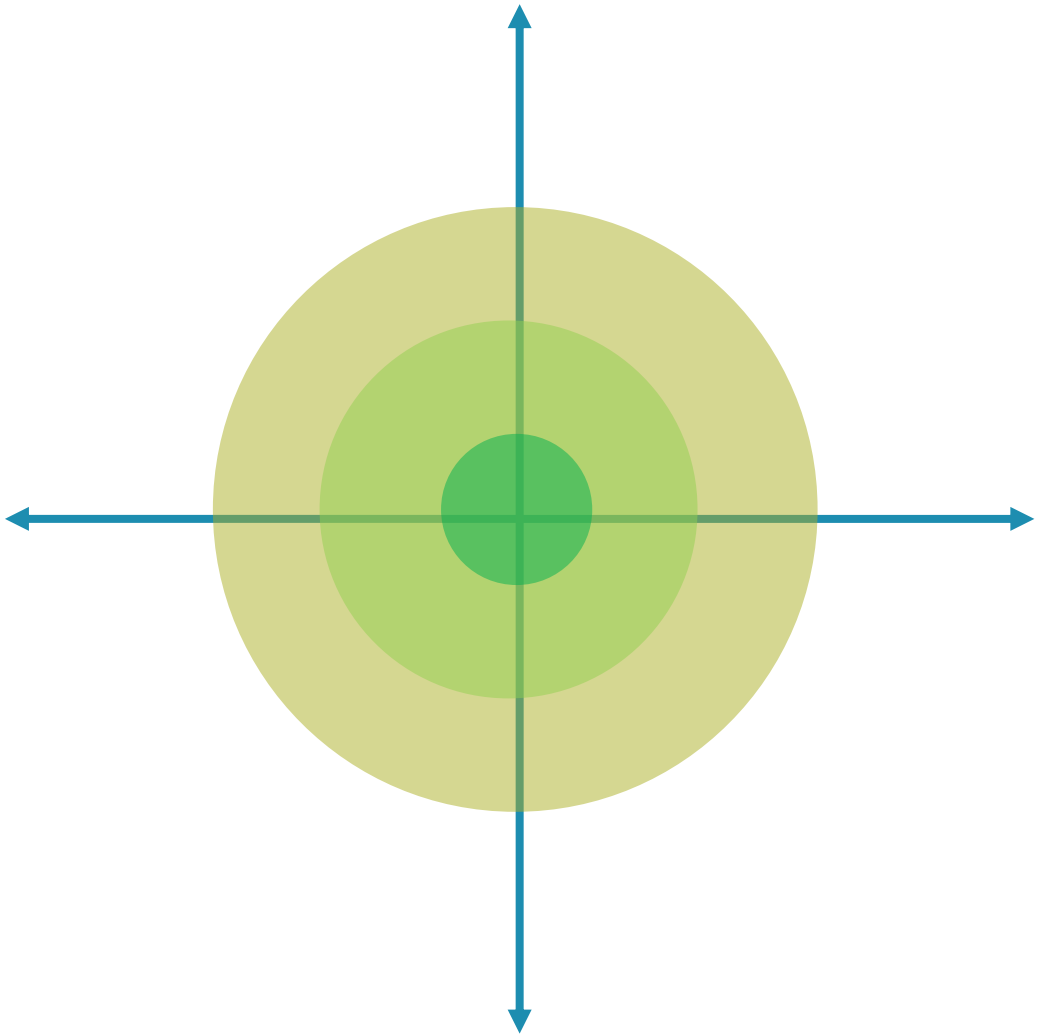


- Large margins mean more weak learners agree, hence more rounds does not necessarily imply that the final classifier is getting more complex
- **Theorem:** Large margins yield better bound on generalization error

Gradient Descent

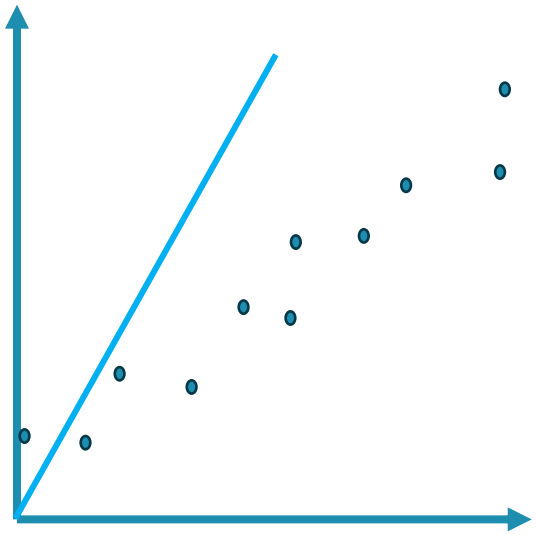
– Continuous search method



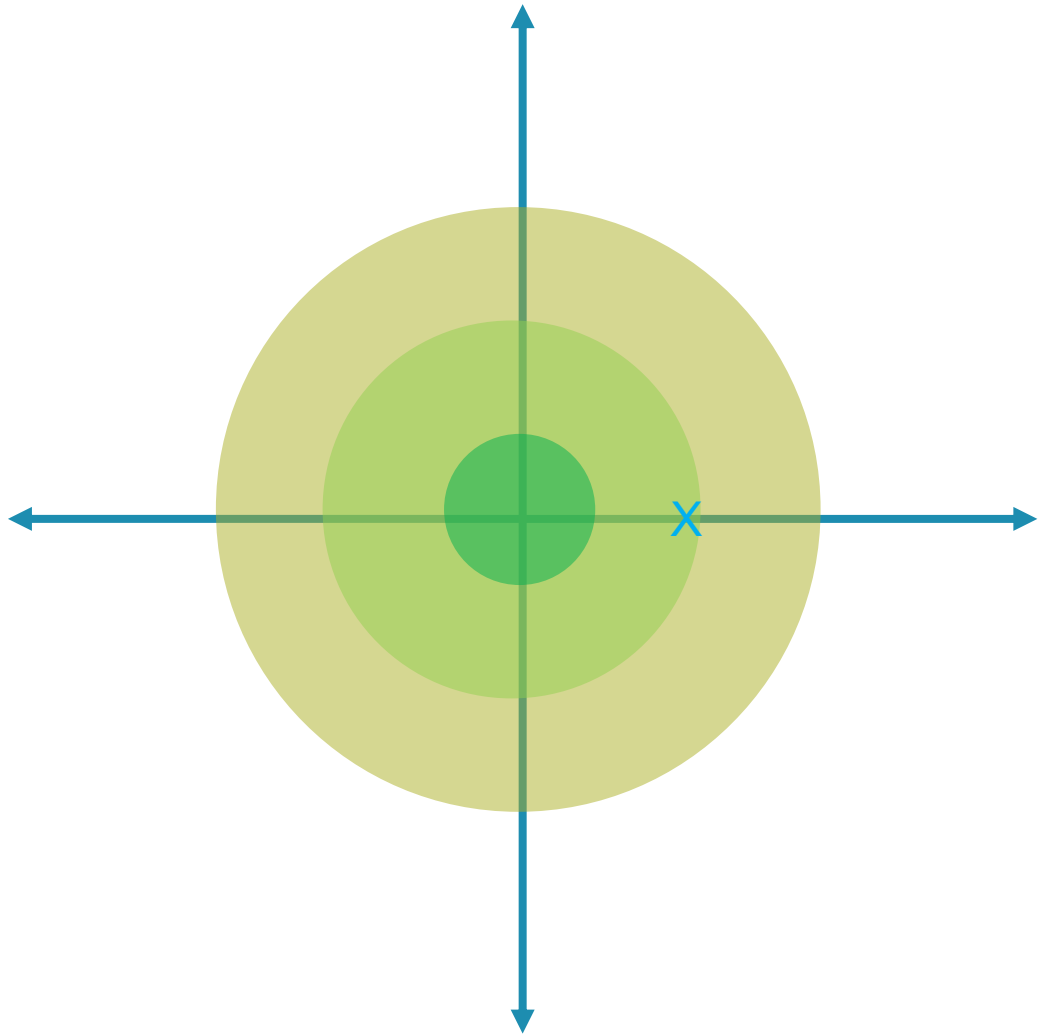


Gradient Descent

– Continuous search method

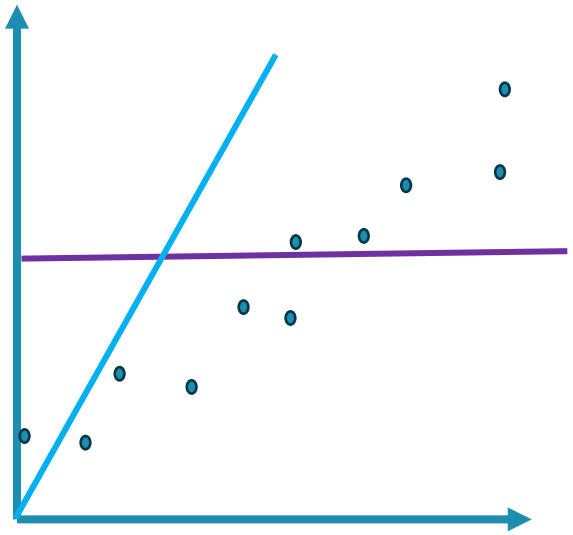


y=2x

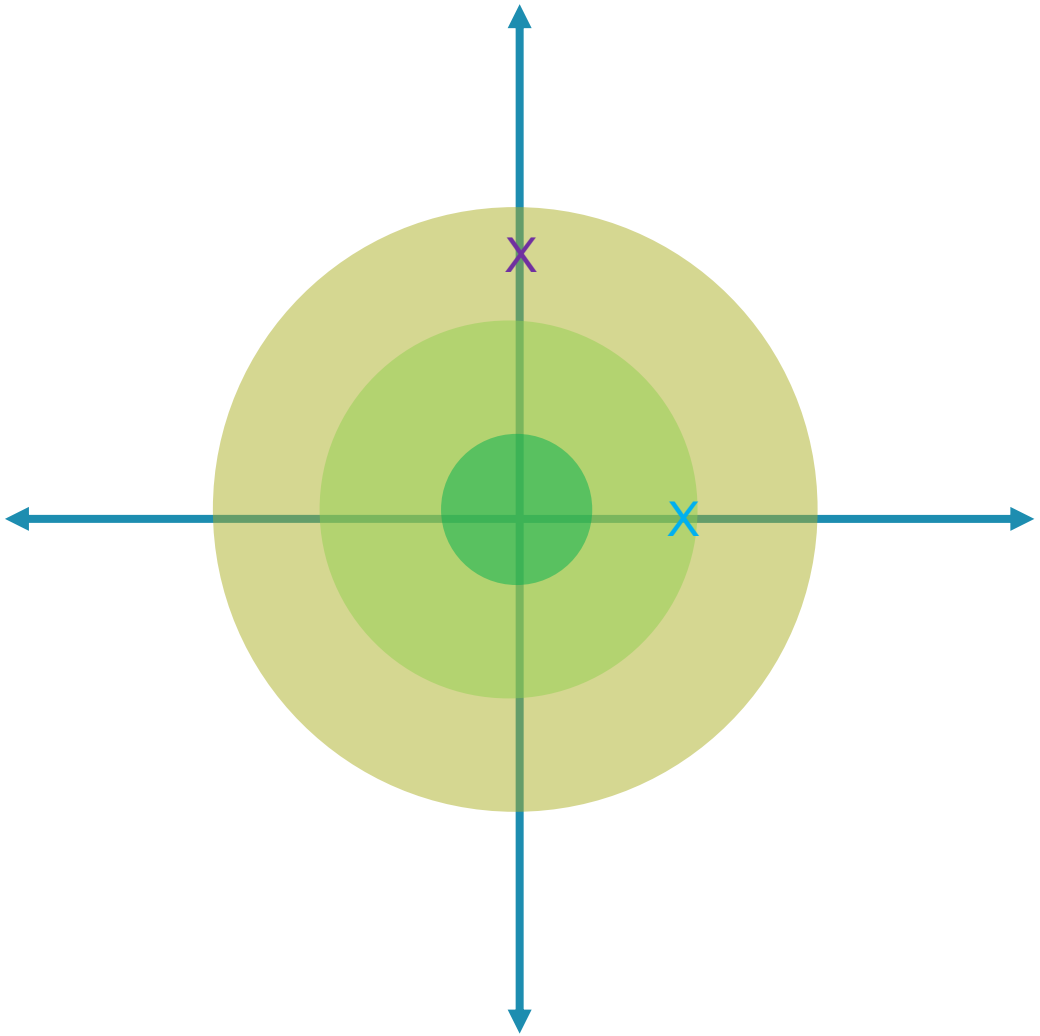


Gradient Descent

– Continuous search method

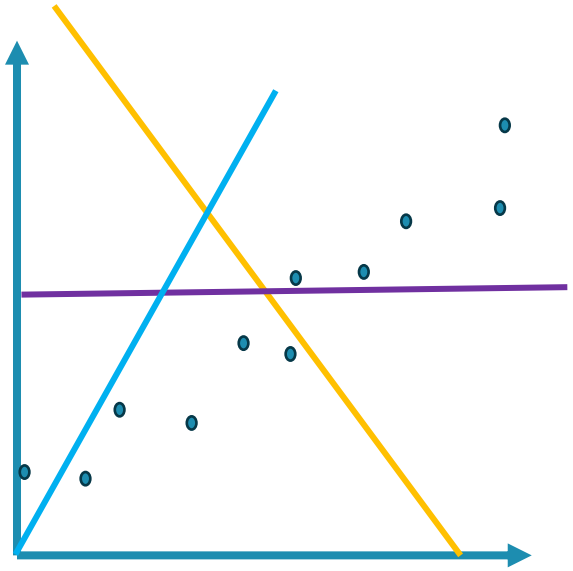


y=2x y=4

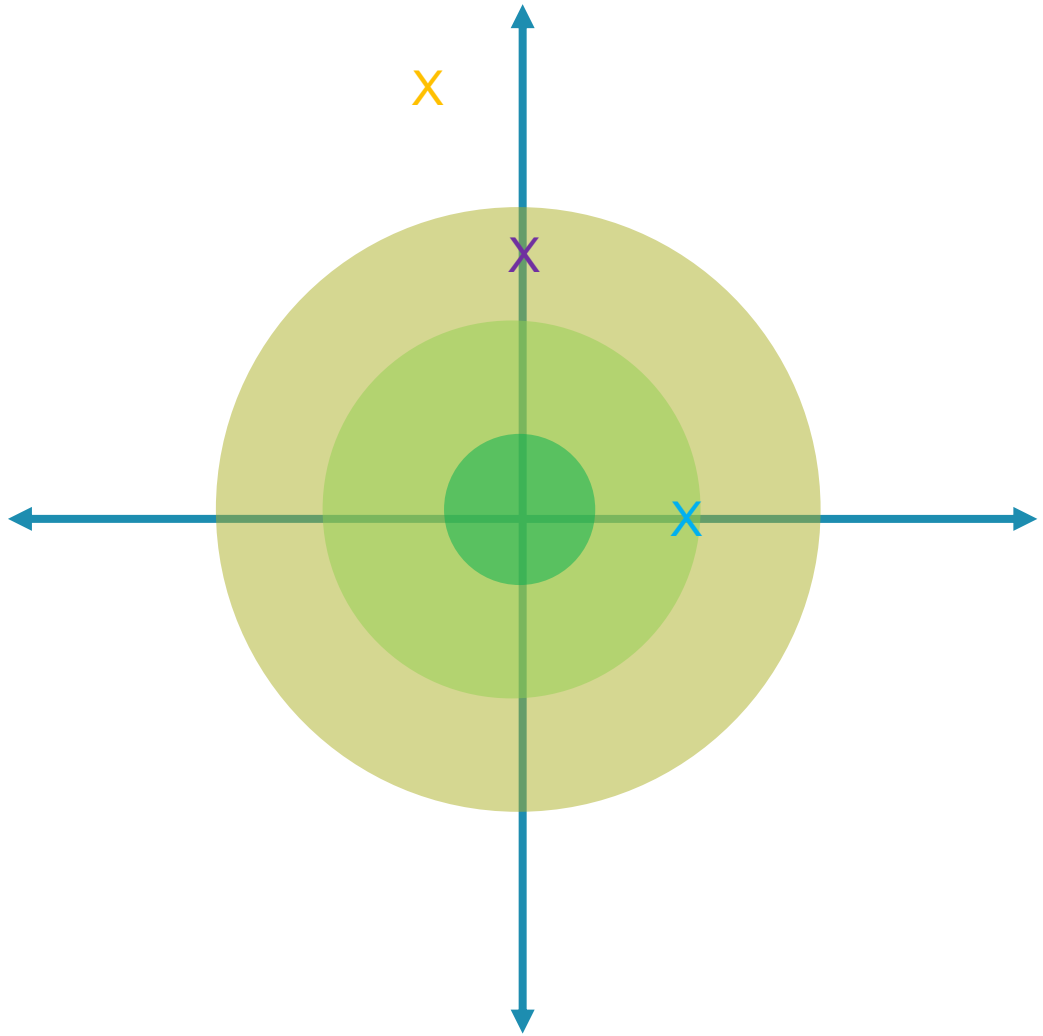


Gradient Descent

– Continuous search method

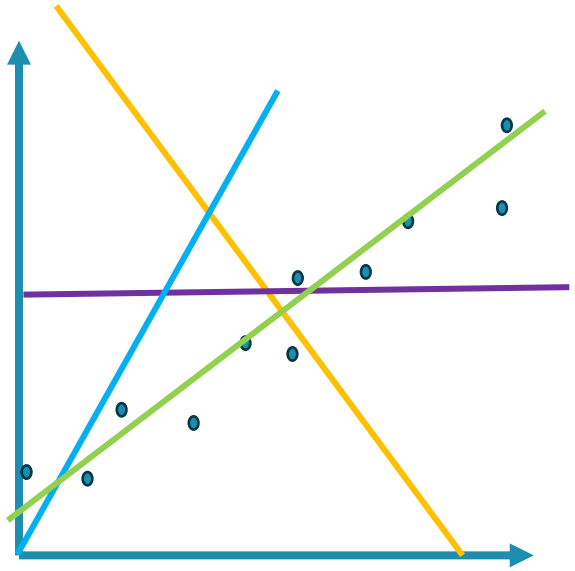


y=2x y=4 y=-x+10

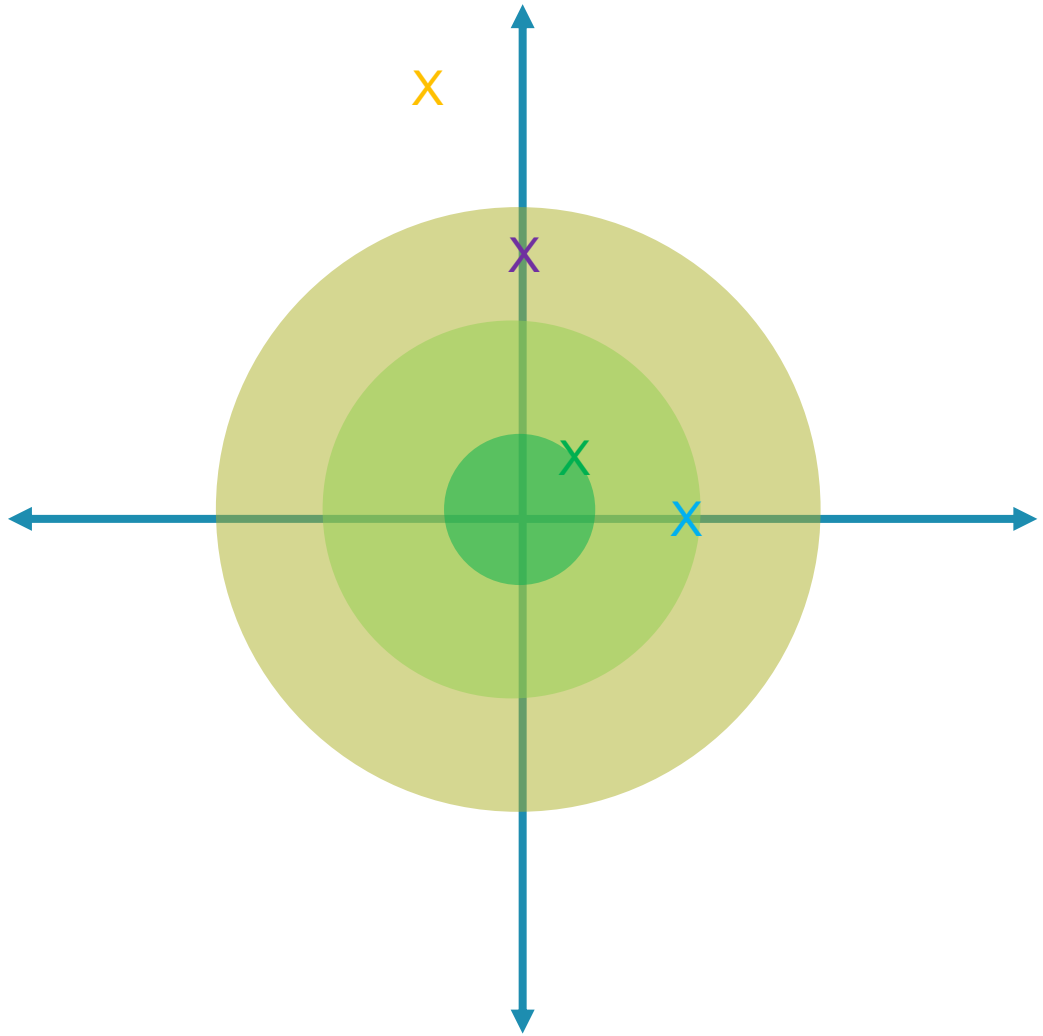


Gradient Descent

– Continuous search method

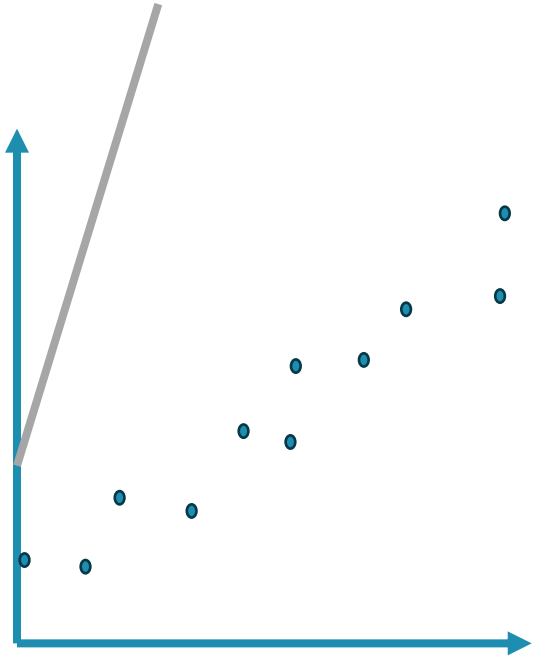


y=2x y=4 y=-x+10 y=1.5x+2

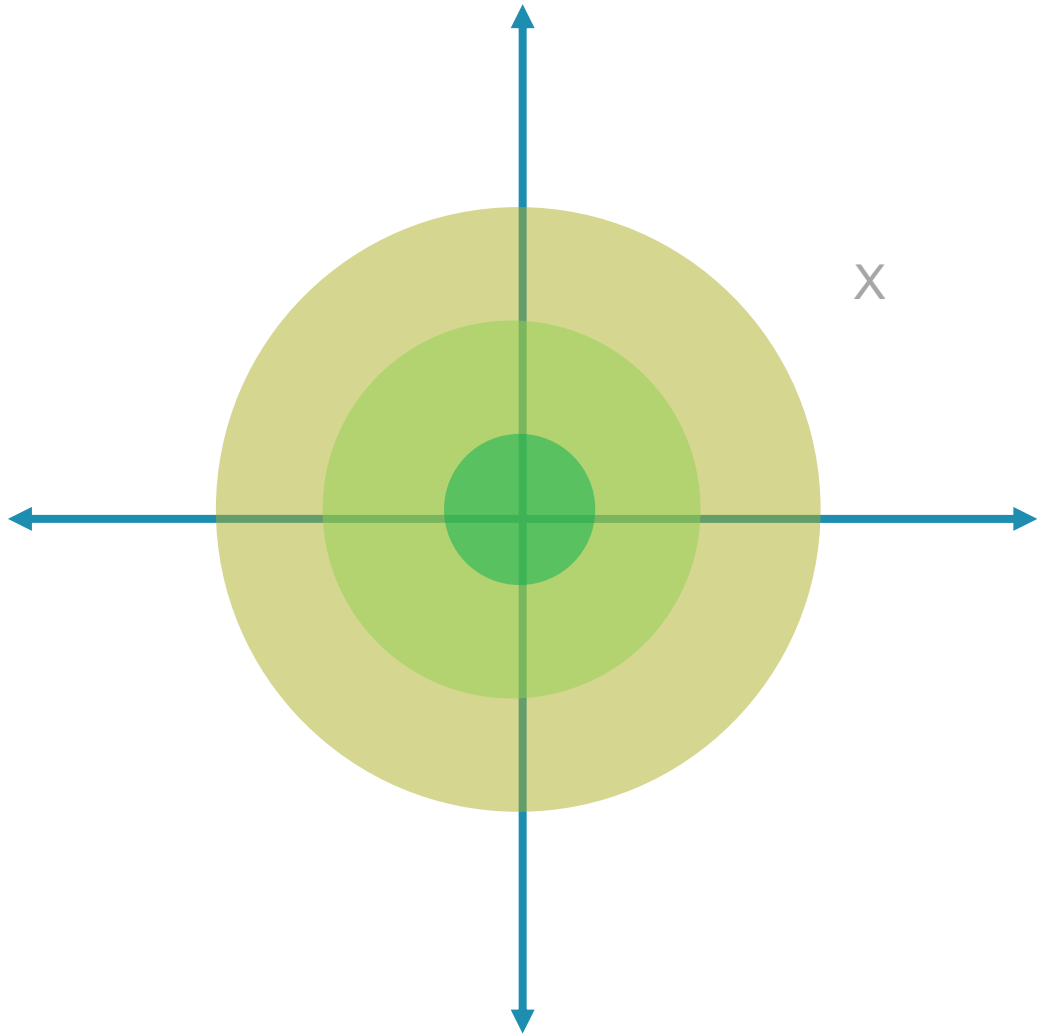


Gradient Descent

– Continuous search method

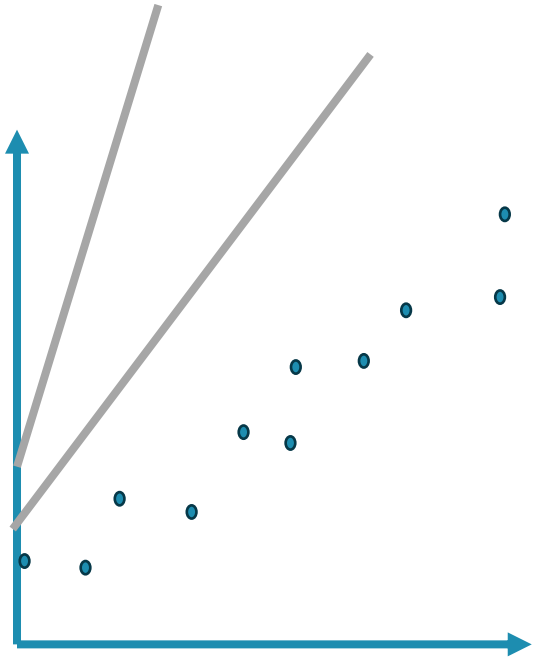


y=2x y=4 y=-x+10 y=1.5x+2

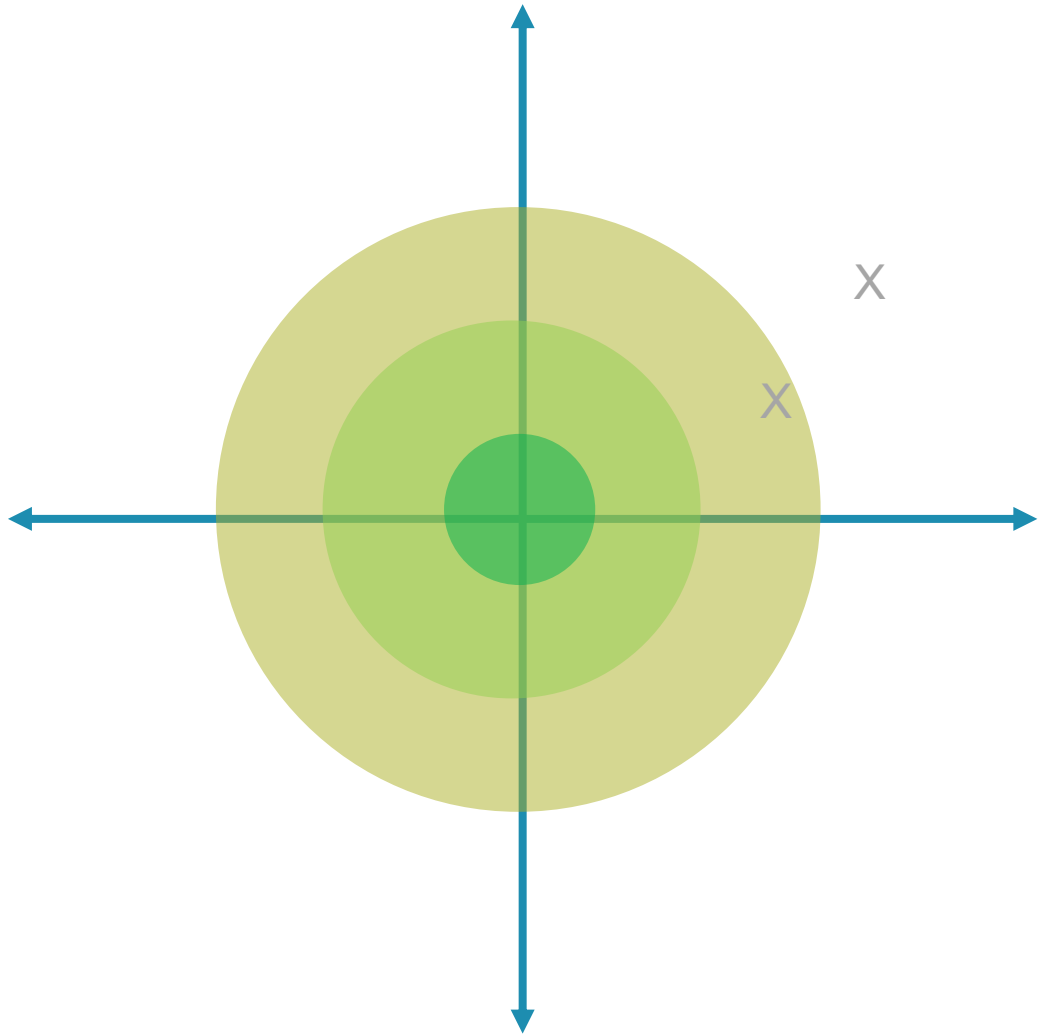


Gradient Descent

– Continuous search method

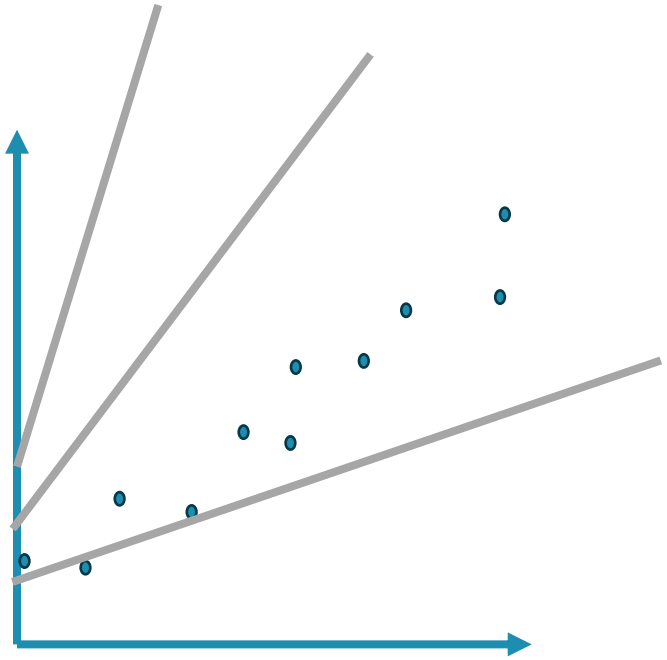


y=2x y=4 y=-x+10 y=1.5x+2

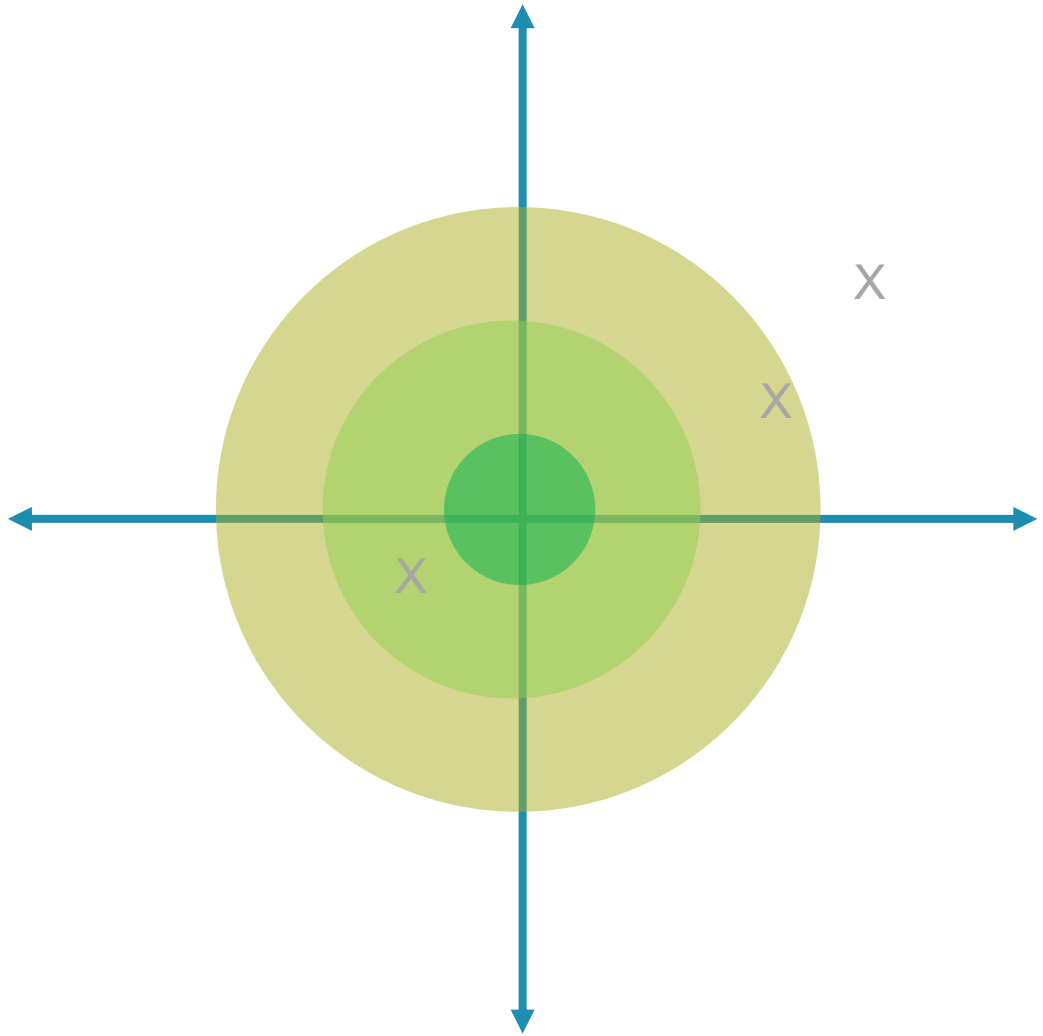


Gradient Descent

– Continuous search method

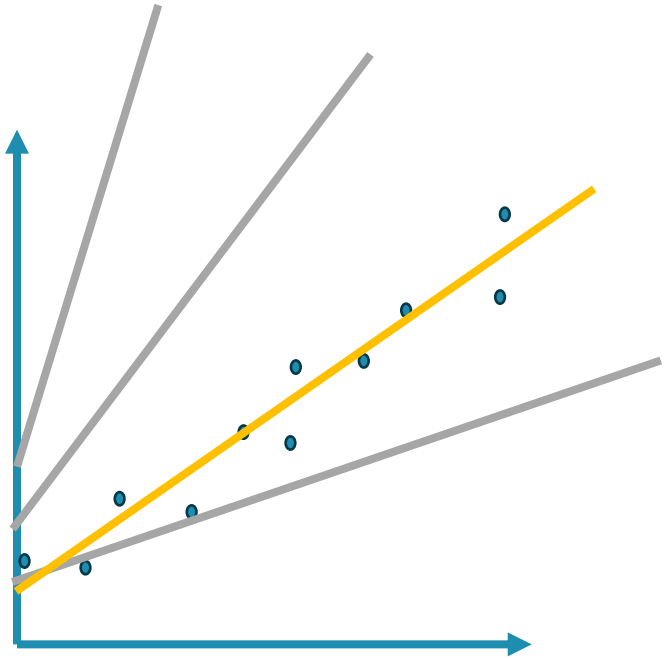


y=2x y=4 y=-x+10 y=1.5x+2

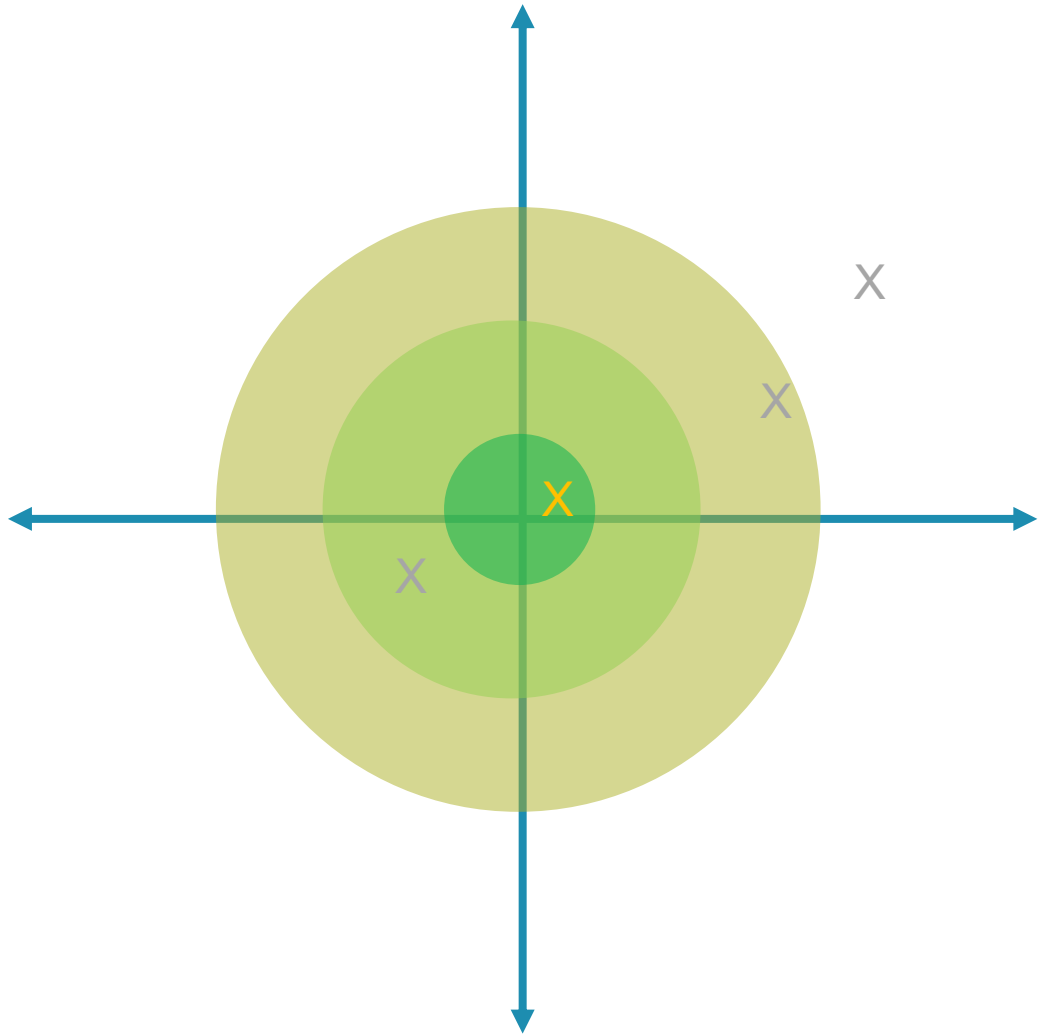


Gradient Descent

– Continuous search method



y=2x y=4 y=-x+10 y=1.5x+2



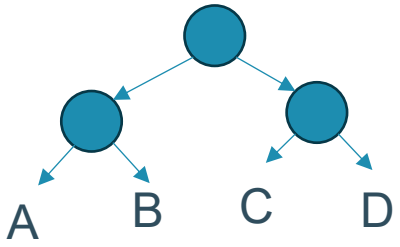
XGBoost

- Extreme gradient boosting
- Gradient boosting: when extending an additive ensemble model $F_n(x) = f_1(x) + f_2(x) + \cdots + f_n(x)$by adding one more component $f_{n+1}(x)$, train $f_{n+1}(x)$ so that adding it represents a step in the direction of the negative gradient
- XGBoost uses decision trees for the f_i functions.
- Currently the gold standard for tabular learning

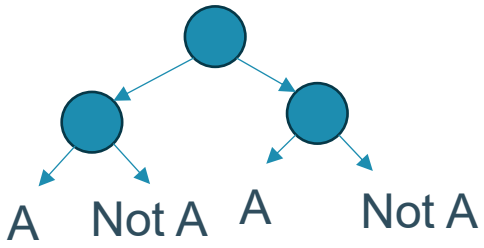
Change the output labels

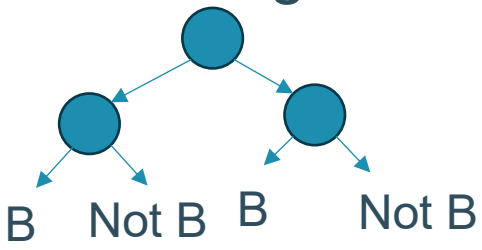
Multiclass problems

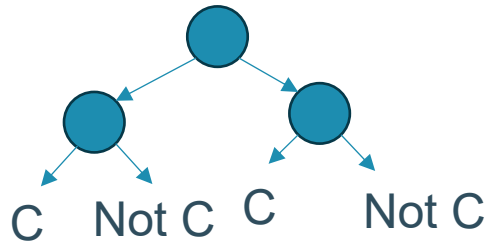
- Given k>2 classes: A B C …
- Some learners can predict multiple classes



- Other option: one-vs-rest. Learn k binary classifiers where
  - One class is positive, the other classes are negative.
  - Take the prediction with the highest confidence







Multiclass problems

- Encode by partition output labels in 2 subsets, create log(k) models
  - A B C D vs E F G H
  - A C E G vs B D F H
  - A B E G vs C D F H

- Could create more than log(k) models to build redundancy: “Error-correcting codes”

Error-Correcting Output Codes

- Ensemble method for multiclass problems
- Build T bit code word for each class
- For class y_k, the i[th] bit is
  - 1 if y_k is new 'pos' class for h_i
  - 0 if y_k is new 'neg' class for h_i
- For t=1..T
  - Partition labels into two disjoint label sets
  - Train model on new label set

Y	Code Words				
	1	2	3	4	5
0	0	0	1	1	0
1	0	1	1	0	1
2	0	0	0	0	1
3	0	1	0	1	0
4	1	0	1	0	1
5	1	1	1	0	0
6	1	0	0	1	0
7	1	1	0	1	1

Error-correcting Output codes

– Given example x, create bit vector $h(x)$ of length T where the ith bit:

  Is 1 if $h_i(x)$ predicts positive

  Is 0 if $h_i(x)$ predicts negative

-Then predict the class with the smallest

Hamming distance to $h(x)$

Error-correcting Output codes

– Given example x, create bit vector $h(x)$ of length T where the ith bit:

Is 1 if $h_i(x)$ predicts positive

Is 0 if $h_i(x)$ predicts negative

-Then predict the class with the smallest Hamming distance to $h(x)$

Code word = [0,1,0,0,0]

Y	Code Words				
	1	2	3	4	5
0	0	0	1	1	0
1	0	1	1	0	1
2	0	0	0	0	1
3	0	1	0	1	0
4	1	0	1	0	1
5	1	1	1	0	0
6	1	0	0	1	0
7	1	1	0	1	1

Y	Hamming Distance
0	3
1	2
2	2
3	1
4	4
5	2
6	3
7	3

Explaining why ensembles work

Expected Error

- Given a new data point x, what is the expected prediction error?
- Assume data points are drawn i.i.d (independent and identically distributed) from a unique underlying probability distribution p

- Goal: Compute for an arbitrary new point x $E_p[(y - h(x)^2)]$
  - E_p is the expectation across all training sets
  - y is the true label
  - $h(x)$ is the predicted label
  - Note: looking at the squared error

Expected error

– Assume that the true function generating the true labels is $f(x)$ and the average prediction of $h(x)$ is $\overline{h(x)}$, then one can decompose

$$E_p[(y - h(x)^2)] =$$

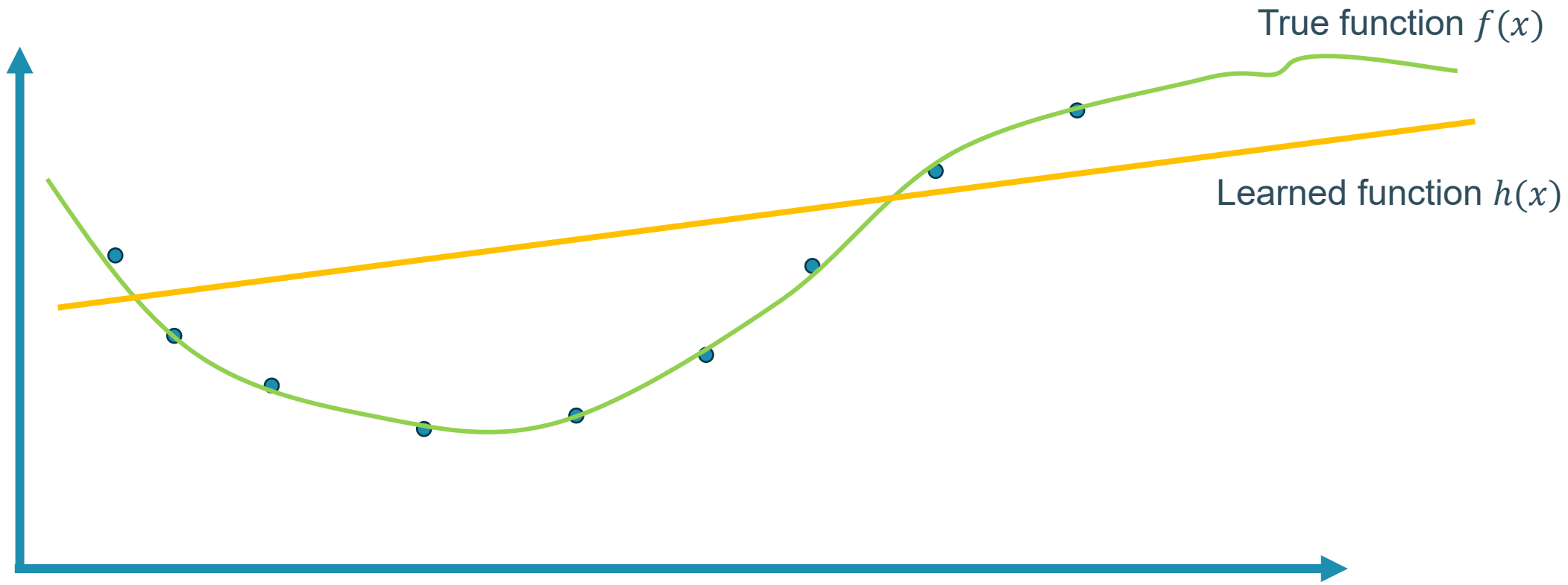
Expected error

– Assume that the true function generating the true labels is $f(x)$ and the average prediction of $h(x)$ is $\overline{h(x)}$, then one can decompose

$$E_p[(y - h(x)^2)] = \underbrace{\left(h(x) - f(x)\right)^2}_{\text{Bias}^2}$$

– **Bias:** Inability to represent the true target

Bias



Sources of bias

- Inability to represent certain decision boundaries
  - E.g., linear threshold units, decision trees
- Incorrect assumptions
  - E.g., naïve Bayes' independence assumption fails
- Classifiers that are "too global"
  - E.g., single linear separator, small decision tree
- If bias is high, the model is underfitting the data

Expected error

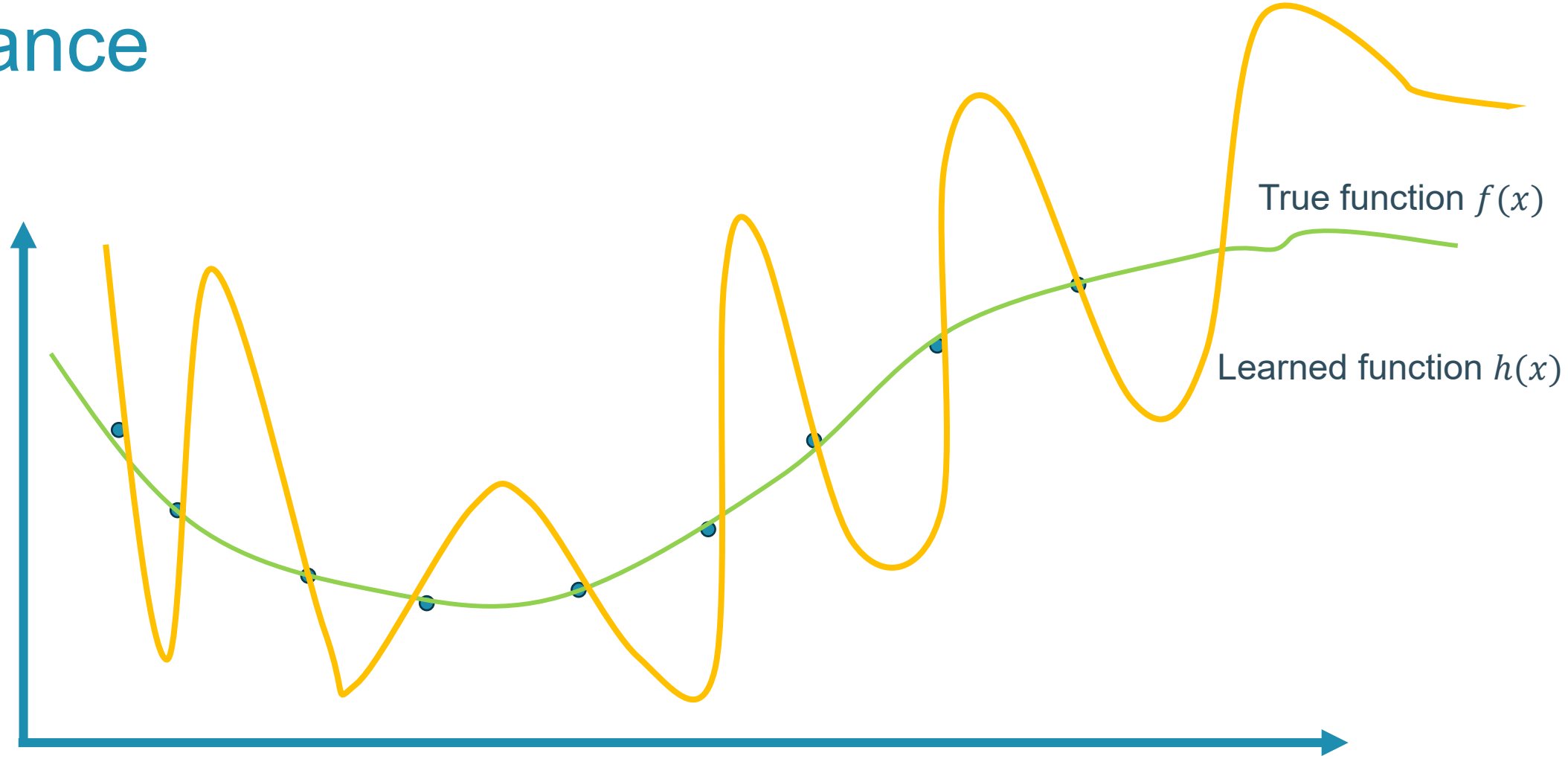
– Assume that the true function generating the true labels is $f(x)$ and the average prediction of $h(x)$ is $\overline{h(x)}$, then one can decompose

$$E_p[(y - h(x)^2)] = \underbrace{\left(h(x) - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathrm{E_p}\left[\left(\mathrm{h(x)} - \overline{h(x)}\right)^2\right]}_{\text{Variance}}$$

– **Bias:** Inability to represent the true target

– **Variance:** Fluctuations due to variations in the data sample

Variance



Sources of Variance

- Statistical sources
  - Classifiers are “too local” and can easily fit the data (e.g., nearest neighbor, large decision trees)
- Computational sources
  - Decision made on small subsets of the data (e.g., decision tree splits near the leaves)
  - Randomization in the learning algorithm (e.g., neural nets with random initial weights)
  - Unstable learning algorithms (e.g., decision boundary can change if one training example changes)
- High variance ⇒ model is overfitting the data

Expected error

– Assume that the true function generating the true labels is $f(x)$ and the average prediction of $h(x)$ is $\overline{h(x)}$, then one can decompose

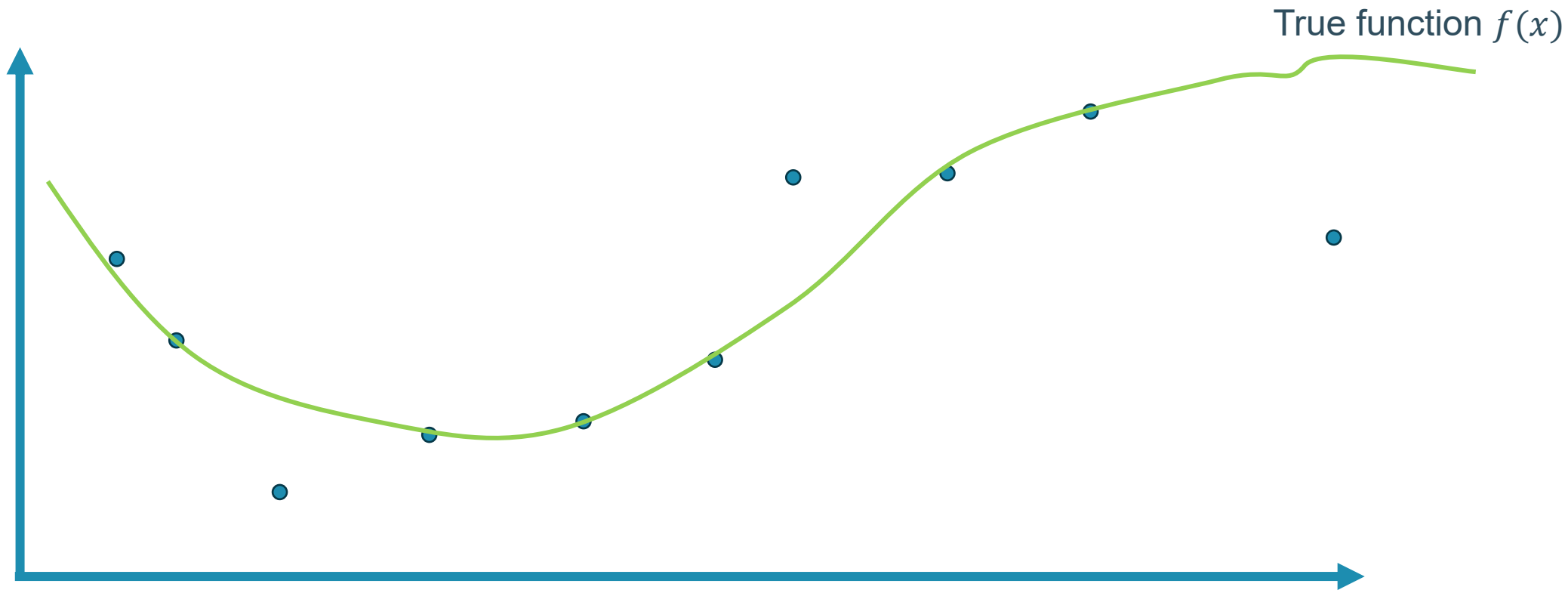
$$E_p[(y - h(x)^2)] = \underbrace{\left(h(x) - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathrm{E_p}\left[\left(\mathrm{h(x)} - \overline{h(x)}\right)^2\right]}_{\text{Variance}} + \underbrace{\mathrm{E_p}[\left(\mathrm{y} - \mathrm{f(x)}\right)^2]}_{\text{Noise}}$$

– **Bias:** Inability to represent the true target

– **Variance:** Fluctuations due to variations in the data sample

– **Noise:** Inability to distinguish between two objects with different labels

Noise



Sources of Noise

- Inherent randomness
  - Some processes are genuinely stochastic, such as equipment failure or customer behavior.
- Unobserved variables
  - Important information is missing from the input features
- Measurement error
  - Sensors, laboratory measurements or data-entry processes record imperfect values.
- Ambiguous or inconsistent labels
  - Different experts may assign different labels to the same example.

Bias variance tradeoff

- Typically:
  - **Bias**: Do not have good hypotheses in the considered class
  - **Variance**: hypothesis class too big
- Hence, we are faced with a trade-off:
  - More expressive hypothesis class, which generates higher variance
  - Less expressive class, which generates higher bias

Explanation 1: Bias-Variance

- Ensembles can address both bias and variance!
- Bagging
  - If bootstrap approximation is correct, then would reduce variance without changing bias
  - In practice, bagging can reduce both
    - For high-bias classifiers, can reduce bias
    - For high-variance classifiers, can reduce variance
- Boosting
  - Early iterations: primarily reduces bias
  - Later iterations: primarily reduces variance (apparently)

Bayesian Estimate

Coin	$P(H\|C_i)$	$P(C_i)$
C_1	0.1	0.05
C_2	0.5	0.25
C_3	0.9	0.7

- 3 possible coins
- We pick a coin (based on $P(C_i)$) , flip it twice and observe Heads - Tails
- We want to predict the third flip → P(H|HT)?

Bayesian Estimate

Coin	$P(H\|C_i)$	$P(C_i)$
C_1	0.1	0.05
C_2	0.5	0.25
C_3	0.9	0.7

- 3 possible coins
- We pick a coin (based on $P(C_i)$) , flip it twice and observe Heads - Tails
- We want to predict the third flip → $P(H|HT)$?
- $P(HT|C_1)=0.1 \times 0.9 = 0.09$; $P(HT|C_2)=0.5 \times 0.5 = 0.25$; $P(HT|C_3)=0.9 \times 0.1 = 0.09$

Bayesian Estimate

Coin	$P(H|C_i)$	$P(C_i)$
C_1	0.1	0.05
C_2	0.5	0.25
C_3	0.9	0.7

- 3 possible coins
- We pick a coin (based on $P(C_i)$) , flip it twice and observe Heads - Tails
- We want to predict the third flip → P(H|HT)?
- $P(HT|C_1)=0.1 \times 0.9 = 0.09$; $P(HT|C_2)=0.5 \times 0.5 = 0.25$; $P(HT|C_3)=0.9 \times 0.1 = 0.09$
- Using Bayes' rule, we combine the prior probabilities ($P(C_i)$) with the likelihood of observing HT: $P(C_i|HT) \propto P(HT|C_i)P(C_i)$
- After normalization: $P(C_1|HT)=0.035$; $P(C_2|HT)=\mathbf{0.481}$; $P(C_3|HT)=\mathbf{0.485}$

Bayesian Estimate

Coin	$P(H|C_i)$	$P(C_i)$
C_1	0.1	0.05
C_2	0.5	0.25
C_3	0.9	0.7

- 3 possible coins
- We pick a coin (based on $P(C_i)$) , flip it twice and observe Heads - Tails
- We want to predict the third flip → P(H|HT)?
- $P(HT|C_1)=0.1 \times 0.9 = 0.09$; $P(HT|C_2)=0.5 \times 0.5 = 0.25$; $P(HT|C_3)=0.9 \times 0.1 = 0.09$
- Using Bayes' rule, we combine the prior probabilities ($P(C_i)$) with the likelihood of observing HT: $P(C_i|HT) \propto P(HT|C_i)P(C_i)$
- After normalization: $P(C_1|HT)=0.035$; $P(C_2|HT)=\mathbf{0.481}$; $P(C_3|HT)=\mathbf{0.485}$

➔We don't know which coin we observed, can we combine them though?

Bayesian Estimate

Coin	$P(H|C_i)$	$P(C_i)$
C_1	0.1	0.05
C_2	0.5	0.25
C_3	0.9	0.7

- 3 possible coins
- We pick a coin (based on $P(C_i)$) , flip it twice and observe Heads - Tails
- We want to predict the third flip → P(H|HT)?
- $P(HT|C_1)=0.1 \times 0.9 = 0.09$; $P(HT|C_2)=0.5 \times 0.5 = 0.25$; $P(HT|C_3)=0.9 \times 0.1 = 0.09$
- Using Bayes' rule, we combine the prior probabilities ($P(C_i)$) with the likelihood of observing HT: $P(C_i|HT) \propto P(HT|C_i)P(C_i)$
- After normalization: $P(C_1|HT)=0.035$; $P(C_2|HT)=\mathbf{0.481}$; $P(C_3|HT)=\mathbf{0.485}$

➔We don't know which coin we observed, can we combine them though?

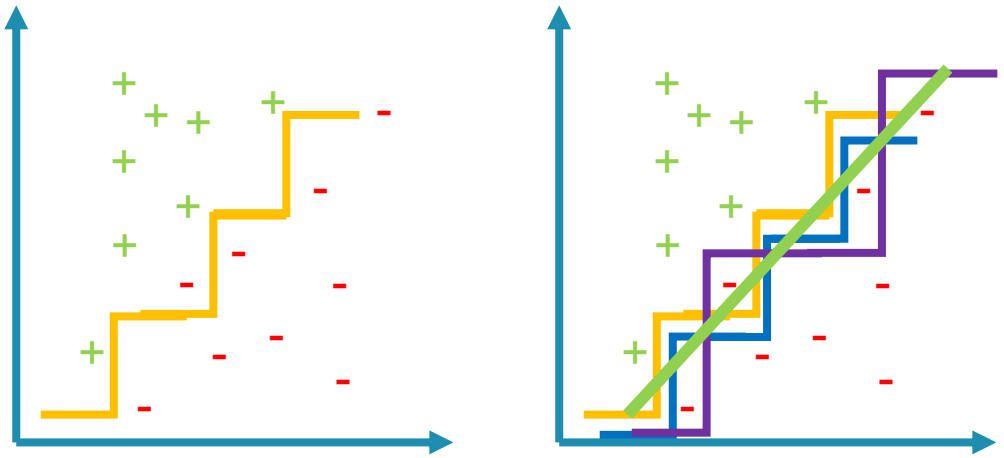
$$P(H|HT) = \sum_i P(H|C_i)P(C_i|HT) = 0.1 \times 0.035 + 0.5 \times 0.481 + 0.9 \times 0.485 \approx 0.68$$

Explanation 2: Statistical explanation

- How can the learning algorithm select among set of (almost) equally good hypotheses?
- Bayes optimal classifier: Weighted majority vote of all hypotheses
  - Weighted by their posterior probability
  - Provably the best possible classifier, that is, minimizes the error rate
- Ensemble learning approximates Bayes optimal

Explanation 3: Representation explanation

- Each learner works with a given hypothesis class
- Possible that the target function lies outside of the considered hypothesis class
- Ensemble averaging may approximate the true target function with arbitrarily good accuracy



Explanation 4: Computation explanation

- Most learning algorithms search through hypothesis space to find one “good” model
- Most interesting hypothesis spaces are:
  - Huge/infinite
  - Heuristic search is essential
- Learner might get stuck in a local minimum
- One strategy for avoiding local minima: Repeat the search many times with random restarts: This is similar to bagging!

Applications

Netflix

- The Netflix Prize Competition
  - Dataset: 100 million film ratings, 480.000 users, 17.700 films
  - Reduce the error of Netflix' Cinematch system by 10%
  - Won by a huge ensemble model
    - Neighborhood models (kNN)
    - Bagged Gradient Boosted Decision tree
    - Temporal models
    - Matrix factorization
    - All combined through stacking

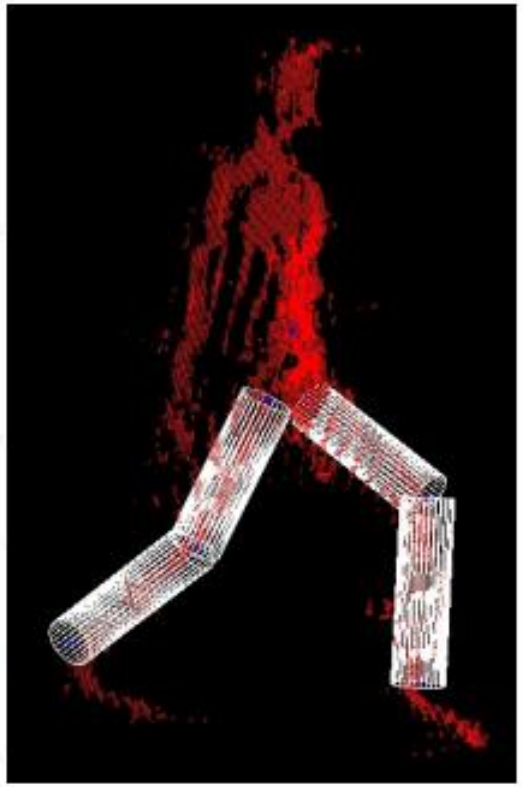
Spotify

- Contextual ranking through XGBoost based on:
  - Collaborative signals: what people with similar listening patterns enjoy.
  - Content signals: properties of the track, such as audio or semantic information.
  - User-history signals: long-term preferences and recent listening.
  - Contextual signals: time, device, session and current activity.
  - Exploration systems: unfamiliar artists or newly released content

Motion capture







jesse davis

depth image ➡ body parts ➡ 3D joint proposals

Shotton et al., CVPR 2011

Questions?